Operating Systems

ASSIGNMENT 1

CO1: *Apply the concepts of operating system types, structures and system calls to demonstrate their functions in various computing environments – history of operating systems, mainframe, desktop, multiprocessor, distributed, clustered, real-time and handheld systems, OS structure, services, system calls, system design, CPU states, memory hierarchy and I/O systems.*

Assignment Title: Evaluate and Justify the Operating-System Design of a Smart Hospital (Total Marks - 50)

Nature of Assignment: Individual
Duration: 1 Week
Submission Date: _____

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

- **Every student must individually execute every task in this assignment**, including all "Complete the Code" programs.
- Create a GitHub repository named **CSA04-<RegisterNumber>-Assignment1** and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C source files, shell scripts, configuration files (Dockerfile, YAML), command outputs and screenshots.
- Include a README.md with your name, register number, compiler and version (for example gcc 13), the Linux distribution and version used, the exact gcc command used to compile (for example gcc -std=c11 -Wall -Wextra -pthread), the steps to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. **An assignment submitted without a working GitHub link will not be evaluated.**

GitHub Repository Link:https://github.com/rakshanasudhakar1905/CSA0409-Operating-Systems/tree/main/ASSIGNMENT

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output (or result) with the expected output.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Scenario — Operating Systems for a Smart Hospital

A 500-bed hospital is replacing its computer systems. It needs: bedside patient monitors that must raise an alarm within 10 milliseconds of an abnormal reading; nurses' handheld tablets; doctors' desktop workstations; a central database server for patient records that must never stop; an imaging server that processes MRI scans using many processors; and a link to two partner hospitals that share records.

The IT team has proposed using one general-purpose operating system everywhere. The clinical staff are concerned that alarms may be delayed, the database may stop during updates, and the tablets' batteries will not last a shift.

The number of devices may grow:

1 ward (40 monitors, 10 tablets) → 1 hospital (500 monitors, 120 tablets, 80 workstations) → a group of 3 hospitals sharing records

You are required to evaluate the requirements of every part of the system, judge which type and structure of operating system suits each, examine how programs obtain OS services through system calls, and defend a complete operating-system plan for the hospital.

Design Challenge:

The hospital is adding remote patient monitoring at home and a cloud-based records service.

Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 1 – Evaluate the Requirements of Each Computing Environment 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate what each part of the hospital needs from its operating system.

Discuss:

- Response-time, reliability, throughput, battery and security requirements of each device
- Which requirements conflict (for example: fast response and many users on one machine)
- Which requirements are critical to patient safety
- How the lessons from the history of operating systems (batch, time-sharing, personal, mobile) apply to each environment

You are expected to **rank the requirements of each environment and justify the ranking**.

Alternative Concept and Complexity Analysis

If the whole hospital used a single general-purpose operating system on every device, instead of a different type of OS for each environment, what would the solution be? Evaluate the worst-case alarm response time (unbounded scheduling delay versus a guaranteed deadline), the memory needed on each device, and the administration effort.

Evidence Expected

A ranked requirement table for every environment with a justification for each ranking.

TASK 2 – Judge the Type of Operating System for Each Environment 3 Marks

Bloom's Level: K5 – Evaluate

Compare the types of operating systems for the hospital.

Discuss:

- Mainframe, desktop, multiprocessor, distributed, clustered, real-time (hard and soft) and handheld systems
- Which type best fits the monitors, tablets, workstations, database server, imaging server and partner link
- Symmetric versus asymmetric multiprocessing for the imaging server
- A clustered system with fail-over for the database server

Alternative Concept and Complexity Analysis

If the database server used a single large multiprocessor machine instead of a cluster of smaller machines with fail-over, what would the solution be? Evaluate the availability (a single point of failure versus fail-over within seconds), the cost, and how throughput grows with the number of processors (time) and the memory available (space).

Evidence Expected

A justified choice of operating-system type for each environment, with the rejected alternatives and reasons.

TASK 3 – Evaluate Operating-System Structures 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate the possible structures of the operating system.

Discuss:

- Simple (monolithic), layered, microkernel, modular (loadable kernel modules) and hybrid structures
- The effect of each structure on reliability, performance and ease of updating
- Which structure suits the patient monitors and which suits the database server
- How Linux, Windows, Android and a real-time OS such as FreeRTOS or QNX are structured

Alternative Concept and Complexity Analysis

If the bedside monitors used a monolithic kernel instead of a microkernel, what would the solution be? Evaluate the time cost of a service request (one system call versus message passing between servers), the memory taken by the kernel, and the effect of a faulty driver on the whole system.

Evidence Expected

A comparison of structures against the hospital's requirements, with a defended choice for two environments.

TASK 4 – Evaluate the Services an Operating System Must Provide 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate the services each environment needs.

Discuss:

- User interface, program execution, I/O operations, file systems, communication, error detection, resource allocation, accounting, protection and security
- Which services can be removed from the bedside monitor to make it smaller and faster
- Which services are essential for the database server
- How the OS protects one patient's records from another user's programs

Alternative Concept and Complexity Analysis

If the tablets used a full desktop operating system with every service enabled instead of a reduced handheld OS, what would the solution be? Evaluate the memory and storage used by the extra services (space), the start-up time and the battery life (time).

Evidence Expected

A service table for each environment with justified inclusions and exclusions.

TASK 5 – Examine System Calls — 4 Marks

Bloom's Level: K5 – Evaluate

Examine how a program obtains OS services.

Discuss:

- The categories of system calls (process control, file management, device management, information maintenance, communication, protection)
- What happens step by step when a program calls read(): library call, trap, mode switch, kernel work, return
- How parameters are passed to the kernel (registers, a block in memory, the stack)
- How strace shows the system calls made by a program

Alternative Concept and Complexity Analysis

If programs called the C library functions fopen, fread and fwrite instead of the open, read and write system calls directly, what would the solution be? Evaluate the number of system calls for a file of n bytes (about n / 4096 with library buffering versus n / b for a buffer of b bytes) and the extra buffer space used by the library.

Evidence Expected

A step-by-step explanation of a system call with strace evidence from a real program.

TASK 6 – Evaluate the Cost of System Calls — 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate how the number of system calls affects performance.

Discuss:

- Why each system call costs much more than an ordinary function call
- How the buffer size changes the number of read() and write() calls when the database copies a file

- How library buffering (fread, fwrite) reduces the number of system calls
- The best buffer size for the hospital's backup program, using your measurements

Complete the Code

Complete TODO 1 below using the read() and write() system calls, and execute the program on Linux.

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

/* Copy src to dst with the read() and write() system calls, using a buffer of
   bufsize bytes. Returns the number of read() calls made, or -1 on error. */
long copy_file(const char *src, const char *dst, size_t bufsize)
{
    char buf[8192];
    int in = open(src, O_RDONLY);
    if (in < 0) return -1;
    int out = open(dst, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (out < 0) { close(in); return -1; }
    long calls = 0;
    ssize_t n;
    // TODO 1: Loop: read up to bufsize bytes into buf (count every read call in calls);
    //   stop when read returns 0 (end of file) or -1; write exactly the n bytes that
    //   were read

    close(in);
    close(out);
    return calls;
}

int main(void)
{
    FILE *f = fopen("records.dat", "w");     /* a 10,000-byte test file */
    for (int i = 0; i < 10000; i++) fputc('A' + i % 26, f);
    fclose(f);

    size_t sizes[] = {1, 512, 4096};
    for (int i = 0; i < 3; i++)
        printf("buffer %4zu bytes: %5ld read() calls\n", sizes[i],
               copy_file("records.dat", "copy.dat", sizes[i]));
    unlink("records.dat");
    unlink("copy.dat");
    return 0;
}
```

Expected output:

```
buffer    1 bytes: 10001 read() calls
buffer  512 bytes:    21 read() calls
buffer 4096 bytes:     4 read() calls
```

Run the program under strace -c and compare the number of read() calls reported with the program's output.

Alternative Concept and Complexity Analysis

If the backup program used memory-mapped files (mmap) instead of read() and write() with a buffer, what would the solution be? Evaluate the number of system calls (a few versus about n / b), the copying between kernel and user space, and the memory used for very large files.

Evidence Expected

Measured system-call counts and times for different buffer sizes, with a justified recommendation.

TASK 7 – Evaluate Process Control System Calls — 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate how a command shell on the database server starts and manages programs.

Discuss:

- What fork(), exec(), wait() and exit() each do
- How a shell runs a command, and how it obtains the command's exit status
- What a zombie and an orphan process are, and how the OS deals with them
- Why fork() followed by exec() is used instead of a single "create process" call

Complete the Code

Complete TODO 1 below using fork(), execlp() and waitpid(), and execute the program on Linux.

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    printf("shell: running job 1\n");
    fflush(stdout);                          /* empty the buffer before fork() */
    // TODO 1: Create a child with fork(). In the child, replace the program with
    //   execlp("expr", "expr", "6", "*", "7", (char *)NULL) and exit(127) if exec
    //   fails. In the parent, wait for the child with waitpid() and print "job 1 exit
    //   status N" using WEXITSTATUS



    printf("shell: running job 2\n");
    fflush(stdout);
    pid = fork();
    if (pid == 0) _exit(3);                  /* a child that fails with code 3 */
    waitpid(pid, &status, 0);
    printf("job 2 exit status %d\n", WEXITSTATUS(status));
    return 0;
}
```

Expected output:

```
shell: running job 1
42
job 1 exit status 0
shell: running job 2
job 2 exit status 3
```

Explain why stdout is flushed before each fork(). Then remove the second fflush(stdout), change _exit(3) to exit(3), redirect the output to a file and explain why "shell: running job 2" now appears twice.

Alternative Concept and Complexity Analysis

If commands were run with posix_spawn() or with threads instead of fork() followed by exec(), what would the solution be? Evaluate the time to start 1,000 commands, the memory copied or shared (copy-on-write), and the isolation between commands.

Evidence Expected

A working program that runs commands like a shell, with process diagrams and a reasoned evaluation.

TASK 8 – Evaluate CPU States and Protection — 3 Marks

Bloom's Level: K5 – Evaluate

Evaluate how the hardware protects the hospital system.

Discuss:

- User mode and kernel mode, privileged instructions and the mode bit
- Interrupts, traps and the timer interrupt
- What would happen to the monitors if any program could disable interrupts
- How the hardware stops one program from reading another program's memory

Alternative Concept and Complexity Analysis

If the monitors used polling (checking the sensor in a loop) instead of interrupts, what would the solution be? Evaluate the worst-case delay before an abnormal reading is noticed (the polling interval versus the interrupt latency) and the CPU time and power wasted by polling.

Evidence Expected

A reasoned explanation of CPU modes, interrupts and protection, applied to the monitors.

TASK 9 – Evaluate the Memory Hierarchy and I/O Systems — 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate how data moves between storage levels and devices.

Discuss:

- The levels of the memory hierarchy (registers, cache, main memory, SSD, hard disk, network storage) with typical access times and sizes
- Where MRI images, patient records and monitor readings should be kept
- Programmed I/O, interrupt-driven I/O and DMA, and which suits the MRI scanner
- The effect of caching on the speed of record look-up

Alternative Concept and Complexity Analysis

If the MRI scanner transferred images with interrupt-driven I/O (one interrupt per word) instead of DMA, what would the solution be? Evaluate the number of interrupts and the CPU time for a 500 MB image, and the memory buffers needed in each method.

Evidence Expected

A justified data-placement plan and I/O method for each device, using typical access times.

TASK 10 – Measure and Compare 4 Marks

Bloom's Level: K5 – Evaluate

Measure the behaviour of your programs on a Linux system.

Discuss:

- The number of system calls made by your copy program for different buffer sizes (strace -c)
- The time spent in user mode and in kernel mode (the time command)
- The time to create 1,000 processes with fork() and to create 1,000 threads
- How the measurements support or contradict your earlier judgements

Alternative Concept and Complexity Analysis

If threads were created instead of processes for the measurements, what would the result be? Evaluate the creation time for 1,000 threads versus 1,000 processes and the memory used by each, and explain the difference using what each one shares.

Evidence Expected

Measured results in tables, compared with expectations, with explained differences.

TASK 11 – Recommend and Defend the Operating-System Plan 4 Marks

Bloom's Level: K5 – Evaluate

Recommend the complete operating-system plan for the hospital.

Discuss:

- The type and structure of operating system for each environment
- The services, protection and I/O methods each will use
- The limitations and risks of your plan
- Whether the plan remains suitable for the group of three hospitals

Your recommendation must be based on **evidence from your own analysis and measurements**.

Alternative Concept and Complexity Analysis

If the whole system were moved to virtual machines on two large servers, instead of separate physical machines for each environment, what would the solution be? Evaluate the effect on response time for the monitors, the memory and processor overhead of virtualisation, and the effect on reliability.

Evidence Expected

A technically justified plan supported by analysis and measurements.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design 8 Marks

Bloom's Level: K5 – Evaluate

Changed Requirement

The hospital adds **remote monitoring devices in patients' homes** that send readings over the internet, and moves patient records to a **cloud service** shared with the partner hospitals.

Re-evaluate your original design. You may **RETAIN → MODIFY → or REDESIGN** your original solution.

A. Impact Analysis – 2 Marks

Identify which operating-system types, services and protection mechanisms are affected.

Evidence Expected

A clear analysis of how the changed requirements affect the original design.

B. Evaluation of the Original Design – 2 Marks

Evaluate whether your plan can support home devices and cloud records, considering response time, reliability and security.

Evidence Expected

A technically supported evaluation of the original design.

C. Adaptation or Redesign – 2 Marks

Describe the revised plan (for example: an embedded real-time OS on home devices, a distributed system for shared records).

Evidence Expected

An appropriate design response.

D. Evidence-Based Defence – 2 Marks

Defend your decision using evidence and the trade-offs involved.

Evidence Expected

A well-supported final decision integrating evidence and the changed requirements.

MARK DISTRIBUTION

Task	Bloom's Level	Marks
Task 1 – Requirements of Each Environment	K5 – Evaluate	4
Task 2 – Types of Operating Systems	K5 – Evaluate	3
Task 3 – Operating-System Structures	K5 – Evaluate	4
Task 4 – Operating-System Services	K5 – Evaluate	4
Task 5 – System Calls	K5 – Evaluate	4
Task 6 – Cost of System Calls	K5 – Evaluate	4
Task 7 – Process Control System Calls	K5 – Evaluate	4
Task 8 – CPU States and Protection	K5 – Evaluate	3

Task	Bloom's Level	Marks
Task 9 – Memory Hierarchy and I/O	K5 – Evaluate	4
Task 10 – Measurement and Comparison	K5 – Evaluate	4
Task 11 – Final Recommendation	K5 – Evaluate	4
Subtotal		**42**
Task 12 – Day-4 Design Challenge	K5 – Evaluate	**8**
TOTAL		**50**

Submission Requirements

- **GitHub repository link (mandatory)** containing the executed code for every task, including every completed "Complete the Code" program, script and configuration file, with input files, output screenshots and a README.md.
- Your assignment report containing the required evidence for Tasks 1–12, including requirement tables, diagrams, strace and time outputs, measurements and complexity analyses.

Assessment Verification

Students may be asked to explain, execute or modify any part of their code during a brief verification discussion/viva. The student should be able to explain and defend the work submitted as their own.

1. Executive summary and report conventions

1.1 The problem in one paragraph

A 500-bed hospital is replacing its computer systems. The system contains six very different computing environments: bedside patient monitors that must raise an alarm within 10 ms, nurses' handheld tablets that must last a shift on one charge, doctors' desktop workstations, a patient-record database that must never stop, an imaging server that processes MRI scans on many processors, and a secure link to two partner hospitals. The IT team proposed one general-purpose operating system (GPOS) everywhere. Clinical staff fear late alarms, database downtime during updates, and flat tablet batteries. This report evaluates every environment, chooses an operating-system type and structure for each, studies how programs obtain OS services through system calls, **measures** real behaviour on Linux, and defends a complete plan. In Task 12 the plan is re-evaluated when home monitoring devices and a cloud record service are added.

1.2 Headline conclusions

S.No	Conclusion	Main evidence
1	Reject the single-OS proposal. Use a small family of OS types matched to each environment.	A GPOS gives no guaranteed alarm deadline; hard real-time OS gives a bounded one (Task 1, 8).
2	Bedside monitors: hard real-time OS (microkernel, e.g. QNX or a certified RTOS such as SAFERTOS/Zephyr-class) with interrupt-driven sensor handling.	10 ms deadline is a safety limit; polling and unbounded scheduling both fail it.
3	Nurses' tablets: handheld OS (Android-class, hybrid/modular kernel) with managed devices.	Battery and security dominate; a full desktop OS wastes memory and battery.
4	Doctors' workstations: desktop/personal OS (Windows or Linux desktop), centrally managed.	Interactive soft response; security and manageability dominate.
5	Database: 2-node (or 3-node) cluster with fail-over on a modular monolithic server OS (Linux/UNIX).	Availability: fail-over in seconds versus a single point of failure.
6	Imaging: symmetric multiprocessing (SMP) server running Linux with DMA-driven I/O.	Throughput scales with cores; DMA removes per-word interrupts.
7	Partner link: distributed system with authenticated, encrypted messaging over a gateway.	Security and interoperability dominate.
8	Our measurements show that system-call count falls from 16,777,216 to 16 (for a 16 MiB file) when the buffer grows from 1 byte to 1 MiB, and that run time falls from an extrapolated 7.9 s to about 0.0075 s (roughly 600–1,000 times faster).	Task 6 and Task 10 tables.
9	Remote home monitoring and cloud records require a modified plan: an embedded RTOS on home devices, a secure edge gateway, and a distributed cloud-records tier. The core local design is retained; the trust model is redesigned.	Task 12.

1.3 How the evidence was produced

All C programs in this report were written for POSIX/Linux, compiled with gcc and executed. Program outputs shown here are real outputs; they are not typed in from the expected-output sheet. The test platform was:

Item	Value
Operating system	Linux 6.18 (x86-64), running in a sandboxed virtual machine
Processors	1 virtual CPU
Compiler	gcc, -O1/-O2
Timing	clock_gettime(CLOCK_MONOTONIC) and getrusage() (user and system CPU time)

Limitations that matter. (a) The test machine is a virtual machine with one CPU, so absolute times will differ on real hardware and multi-core systems; the *ratios and trends* are the evidence. (b) The strace utility was not installed in the sandbox, so Section 6.5 and Section 7.5 give the exact commands to run and the expected shape of the output, and the system-call counts were obtained by counting the read() calls inside the program itself and by reasoning from the measured buffer sizes. When you run the report's programs on your own Linux machine, paste your own strace -c output into the places marked "Insert your strace output". This honesty is part of the evaluation: evidence is labelled as measured, derived or estimated throughout.

1.4 Labelling of numbers

- **Measured** – produced by running the programs in this report.
- **Typical** – standard textbook or vendor ranges (for example memory access times); they are order-of-magnitude values.
- **Estimate** – a design calculation made for this hospital using stated assumptions.

2. Task 1 – Evaluate the Requirements of Each Computing Environment

2.1 The environments and the assumptions made

The hospital scenario gives six environments. The scale grows from one ward (40 monitors, 10 tablets), to the whole hospital (500 monitors, 120 tablets, 80 workstations), to a group of three hospitals.

Environment	Count (hospital)	What it does	Key assumption used in the analysis
E1 Bedside monitor	500	Samples heart rate, SpO_2, blood pressure, ECG; raises an alarm	Alarm within 10 ms of an abnormal reading; runs 24×7; mains power plus a backup battery
E2 Nurse tablet	120	Charts, medication checks, bedside record look-up	Must last a 12-hour shift; carried around, so it can be lost or stolen
E3 Doctor workstation	80	Browsing records, ordering tests, viewing reports	Interactive use; many applications at once; managed by IT

Environment	Count (hospital)	What it does	Key assumption used in the analysis
E4 Database server	1 service (clustered)	Stores patient records	"Must never stop"; assumed target 99.99% availability (about 52 minutes of downtime per year)
E5 Imaging server	1–2	Processes MRI scans on many processors	Study size from tens of MB to hundreds of MB; many slices are processed in parallel
E6 Partner link	2 partners (3 hospitals in total)	Shares records with partner hospitals	Crosses a network the hospital does not control

2.2 Requirement-by-requirement analysis

The task asks for response time, reliability, throughput, battery and security. Each is defined first, so that the ranking later is meaningful.

- **Response time** – delay between an event and the system's reaction. It can be *hard* (missing the deadline is a failure), *soft* (late is worse but still useful), or *interactive* (a person is waiting).
- **Reliability / availability** – the probability the system works correctly when needed, and the fraction of time it is up.
- **Throughput** – amount of work completed per unit time (records per second, MRI slices per minute).
- **Battery** – energy efficiency, meaning how long the device operates without mains power.
- **Security** – confidentiality, integrity and availability of patient data, and protection from unauthorised programs and users.

E1 – Bedside monitor

A monitor does a very small amount of work, but the work is safety-critical. It samples sensors perhaps 250–500 times per second per ECG channel and compares values with limits. The only thing that matters is that, when a value crosses a limit, the alarm sounds within 10 ms *every single time*, not on average. This is the definition of **hard real time**. The monitor has few users (a nurse touches the screen occasionally), little data, and mains power, so throughput and battery are not significant. Security matters because a compromised monitor could suppress alarms or leak data, but the main mitigation is to give the device a tiny attack surface (few services, few ports).

E2 – Nurse tablet

The tablet is battery-powered, mobile, and carries patient data outside secured rooms. A flat battery in the middle of a medication round is a clinical risk, and a lost tablet with unencrypted records is a privacy breach. Response time is *interactive soft*: a screen should react within about 100 ms, but a 300 ms delay causes no harm. Throughput is low.

E3 – Doctor workstation

Doctors run several programs at once (record viewer, e-mail, imaging viewer). The main requirement is a responsive time-sharing system that is secure and centrally manageable. Battery does not apply. Reliability is important but a single workstation failing is tolerable because another can replace it.

E4 – Database server

The database is the hospital's memory. If it stops, nurses cannot check allergies or medication history. Reliability and availability are therefore the first requirement, and the scenario states this directly ("must never stop"). It also needs high transaction throughput (thousands of look-ups and updates per second as the hospital grows to three sites) and strong protection of one patient's records from other users. Updates are the notable risk: patching a single machine normally requires a restart. Response time is soft (a query should return in milliseconds to a few hundred milliseconds).

E5 – Imaging server

MRI processing is **throughput-bound** and compute-heavy: reconstruction and filtering of hundreds of slices. The server's main requirement is to keep every processor busy and to move large images without wasting CPU on I/O. An occasional crash is undesirable but the study can be re-run from stored raw data, so its reliability is ranked second, not first. Response time is soft or batch-like (a radiologist expects results in seconds to minutes).

E6 – Partner link

The link carries patient records to organisations outside the hospital. Security is the main requirement: authentication, encryption, audit logging. Reliability is next, because a link failure must not corrupt records or stop local care (the hospital must keep working if the partners are unreachable). Throughput and latency are moderate.

2.3 Ranked requirement table

Rank 1 = most important for that environment. The score column explains the rank (weight 5 = critical, 1 = minor).

Environment	Response time	Reliability	Throughput	Battery	Security
E1 Bedside monitor	**1** (weight 5: hard 10 ms)	**2** (5)	5 (1)	4 (2)	3 (4)
E2 Nurse tablet	4 (3: interactive)	3 (4)	5 (1)	**1** (5: 12-hour shift)	**2** (5)

Environment	Response time	Reliability	Throughput	Battery	Security
E3 Doctor workstation	2 (4: interactive)	3 (4)	4 (2)	5 (1: mains)	**1** (5)
E4 Database server	4 (3)	**1** (5: never stop)	3 (4)	5 (1: UPS)	**2** (5)
E5 Imaging server	4 (2)	2 (4)	**1** (5: many CPUs busy)	5 (1)	3 (4)
E6 Partner link	4 (2)	2 (4)	3 (3)	5 (1)	**1** (5)

Justification of the rankings.

- **E1:** The first requirement is the one that defines the product: a 10 ms alarm is a clinical safety limit. Reliability is second because a monitor that crashes is a monitor that cannot alarm. Security is third (it must not be possible to switch off alarms remotely) but cannot outrank the deadline itself. Throughput is last because the data rate is tiny.
- **E2:** Battery is first because a dead tablet cannot do its job at all, and the shift length is a hard physical limit. Security is second because the device is portable, so physical loss is the dominant threat. Response time is only fourth because a short delay does not harm the patient.
- **E3:** Security ranks first because doctors have broad access to records and the workstation is exposed to e-mail and web content. Interactive response is second because slow screens waste expensive clinical time.
- **E4:** Reliability is first, taken directly from the scenario. Security is second because one database holds every patient's records. Throughput is third because it grows with the three-hospital plan.
- **E5:** Throughput is first by definition of the workload. Reliability is second because lost processing time delays diagnoses. Response time is least important because processing is batch-like.
- **E6:** Security is first because data leaves the hospital. Reliability is second because a corrupt or half-sent record is worse than a late one.

2.4 Which requirements conflict?

Trade-offs exist between many pairs. The most important are:

Conflict	Why it arises	Where it matters in the hospital
Fast response vs. many users/programs on one machine	Time-sharing divides the CPU among many processes. With *n* runnable processes, a process may wait for a full cycle of time slices before it runs again. A timing guarantee cannot be given when the CPU is shared with unknown work.	A monitor running a browser or other background services could miss its deadline. This is why monitors run only a small, fixed set of tasks.
Throughput vs. response time	Large time slices and batching improve throughput (fewer context switches) but increase waiting time. Small time slices do the opposite.	Imaging server (favours throughput) versus workstation (favours response).
Security vs. response time	Encryption, authentication, and checks cost CPU time. Memory protection and mode switches add small delays.	Partner link and tablets accept the cost; the monitor must be careful to keep security checks off the alarm path.

Conflict	Why it arises	Where it matters in the hospital
Battery vs. performance	Higher clock speed and constant background activity drain batteries.	Tablets use aggressive power saving and sleep states, which add wake-up latency.
Reliability vs. update speed (and cost)	Updates normally require a restart, which causes downtime. Redundancy (clusters) raises cost and complexity.	Database server needs cluster fail-over so one node can be patched while another serves.
Generality vs. determinism	A GPOS has many features and dynamic behaviour (paging, caching, deferred work) that make worst-case timing unpredictable.	The single-OS proposal fails mostly because of this.

2.5 Requirements critical to patient safety

The requirements can be divided by consequences of failure:

1. **Safety-critical (failure can directly harm a patient):** the 10 ms alarm deadline (E1), the continuous availability of records during care (E4), and the integrity of data (no wrong-patient records, no corrupted medication data) in E4 and E6.
2. **Care-critical (failure disrupts care but a workaround exists):** tablet battery life (E2) because paper or a charger is a workaround; workstation responsiveness (E3).
3. **Business-critical (failure delays work but not care):** imaging throughput (E5), unless the scan is urgent.

This tells the designer where to spend money and effort: hard deadlines and hardware redundancy where safety is at stake, ordinary engineering elsewhere.

2.6 Lessons from the history of operating systems

Operating systems evolved to solve different problems. Each stage maps to part of the hospital.

Stage	Original goal	Key technique	Hospital environment where the lesson applies
Batch systems	Keep an expensive computer busy; no interaction	Jobs grouped and run one after another; spooling	**Imaging server (E5):** MRI reconstruction jobs are queued and run for maximum throughput. Nightly **backups** are also batch jobs.
Multiprogramming and time-sharing	Improve CPU use while jobs wait for I/O; give users quick response	Several jobs in memory; CPU scheduling with time slices; virtual memory	**Doctor workstations (E3)** and the **database server (E4)** that serves many users concurrently.

Stage	Original goal	Key technique	Hospital environment where the lesson applies
Personal computers (desktop)	One user, convenience and responsiveness	GUI, plug-and-play, file systems for individuals	**Workstations (E3)**: interactive, user-friendly; security was added later because early PCs had none.
Mobile / handheld	Portability, touch, limited battery and memory	Power management, app sandboxes, app permissions, wireless	**Tablets (E2).**
Real-time systems	Guaranteed response to events	Priority pre-emptive scheduling, bounded interrupt latency, small kernels	**Bedside monitors (E1).**
Distributed / clustered	Share resources over networks, tolerate failure	Message passing, replication, fail-over	**Database cluster (E4)** and **partner link (E6).**

The central historical lesson is that **there is no single best operating system**: each generation optimised one goal (utilisation, response, convenience, battery, determinism, availability) at the expense of others. The IT team's proposal ignores this lesson by trying to serve all goals with one design.

2.7 Alternative concept and complexity analysis – one general-purpose OS on every device

The task asks: *if the whole hospital used a single general-purpose operating system on every device, what would the solution be?* The analysis compares the proposal against the mixed design along three dimensions.

(a) Worst-case alarm response time

A general-purpose kernel is designed to maximise average throughput and fairness. Delays on the path "sensor interrupt → alarm output" include:

- **Scheduling delay** because the alarm task competes with other runnable tasks. On a fair scheduler, the waiting time is bounded by roughly *(number of runnable tasks) × (time slice)*. With 20 runnable tasks and a 10 ms slice, a task could wait up to about **200 ms** in the worst case; with priority boosts for interactive tasks the real behaviour is more complex but there is still no hard promise.
- **Non-preemptible kernel sections and interrupt-disabled sections** (such as a long driver routine), which can delay a runnable task for milliseconds.
- **Page faults** and swapping: a page fault to an SSD costs on the order of 0.1 ms, to a hard disk around 5–10 ms, and under memory pressure there can be many in a row.

- **Background services**: updates, indexing, antivirus scans and garbage collection can monopolise CPU or I/O for hundreds of milliseconds to seconds.

Therefore the worst-case alarm delay on a general-purpose OS is **unbounded in principle** and in practice ranges from a few milliseconds when idle to hundreds of milliseconds or seconds when loaded. A hard real-time OS provides a **guaranteed deadline**: interrupt latency of tens of microseconds, a bounded context-switch time of a few microseconds, fixed-priority pre-emptive scheduling and no paging. The worst-case time can then be *computed* and shown to be below 10 ms with a large margin (see Task 8: typically well below 1 ms).

Property	Single GPOS	Real-time OS
Worst-case alarm delay	Unbounded; can exceed 10 ms under load	Bounded; calculated, typically under 1 ms
Meets 10 ms deadline?	Only on average, **not guaranteed**	Guaranteed by design and testing
Method of proof	Statistical testing only	Schedulability analysis (e.g. rate-monotonic)

A real-time OS also *can* run in a GPOS only through special real-time extensions (for example a Linux with the PREEMPT_RT patches), which narrow the gap considerably but need careful configuration and certification; this is mentioned again in Task 11 as the fallback option.

(b) Memory needed on each device

The table gives order-of-magnitude **estimates** of what each device would need.

Device	GPOS on every device (RAM/storage)	Matched OS (RAM/storage)	Wasted by GPOS
Bedside monitor	≥ 1–2 GB RAM, 8–20 GB storage	Real-time microkernel: kernel under 1 MB; whole system 16–128 MB RAM and a few hundred MB flash	About 90–99% of RAM; higher part cost and heat
Tablet	4 GB RAM, 20 GB storage (desktop OS)	Handheld OS: 2–4 GB RAM, 8–16 GB storage with a smaller footprint	Several GB of storage; more background processes
Workstation	4–8 GB RAM	same	none
Database / Imaging server	16 GB and more	same	none (servers need large memory anyway)

Across 500 monitors the extra 1–2 GB per device is **0.5–1 TB** of RAM that must be purchased, powered and cooled to hold software that never does anything for the monitor's job. In addition, larger software has a larger **attack surface** (more code means more bugs).

(c) Administration effort

One OS everywhere does reduce *some* effort: one skill set, one patch process, one set of tools. That is the genuine advantage of the IT team's proposal and it should be acknowledged. However, other costs increase:

- Every security patch applies to **every** device, including monitors, which would need to be restarted. A restart of a monitor is not acceptable during care, so patching 500 monitors needs a risk-managed plan anyway.
- A single vulnerability can compromise all ~700 devices at once (a *monoculture* risk), whereas in a mixed system the attacker must find different weaknesses.
- Configuration differs strongly across device roles; a single OS needs hundreds of settings disabled differently on each role.

Aspect	Single GPOS	Mixed OS design
Skills / tools	One family	3–4 families (more training)
Patch streams	1	3–4, each with its own risk window
Monoculture risk	High (one flaw affects all)	Lower (diversity)
Certification (medical device)	Difficult: huge code base	Easier for the small RTOS on monitors
Total administration effort	Lower on paper, higher in practice for monitors	Higher on paper, targeted where needed

3. Task 2 – Judge the Type of Operating System for Each Environment

3.1 The types of operating system compared

The types below are defined by the *problem they solve*, not by the company that sells them. Each is compared with the hospital's needs.

Type	Main idea	Strengths	Weaknesses	Typical use
Mainframe	Very large, very reliable machine running batch and transaction work for thousands of users; hot-swappable hardware	Extreme reliability and I/O capacity, strong accounting and security	Very expensive; proprietary; skills are scarce	Banks, national records
Desktop (personal)	One user at a time, GUI, many applications	Convenient, cheap, huge application support	Weak timing guarantees; exposed to malware	Doctors' workstations
Multiprocessor (parallel)	Two or more CPUs share memory and I/O under one OS	Higher throughput	Needs careful locking; scalability limits	Imaging server

Type	Main idea	Strengths	Weaknesses	Typical use
Distributed	Separate computers cooperate through a network and appear as one system	Resource sharing, scalability, geographic spread	Network delay, partial failure, security exposure	Hospital group; partner link
Clustered	Several computers share storage and monitor each other; if one fails another takes over	High availability, rolling upgrades	Cost; complexity; shared storage becomes critical	Database server
Real-time – hard	A missed deadline is a system failure; bounded latency	Predictable, small, certifiable	Few features; low throughput; specialised skills	Bedside monitor
Real-time – soft	Deadlines matter but a late result is still useful	Better average response than a GPOS	No absolute guarantee	Streaming, telemetry display
Handheld	Small battery-powered device with limited memory and a touch screen	Low power; app sandboxing	Limited resources; vendor-controlled updates	Nurse tablets

3.2 Choice for each environment (with rejected alternatives)

Environment	Chosen OS type	Rejected alternatives and why
E1 Bedside monitor	**Hard real-time** OS (small, pre-emptive, priority-based)	*Desktop/GPOS:* no guaranteed deadline. *Soft real-time:* a missed 10 ms limit is a patient-safety failure, so "usually on time" is not enough. *Handheld:* aims at battery rather than guaranteed latency.
E2 Nurse tablet	**Handheld** OS with mobile-device management	*Desktop OS:* too much memory, too short battery life, no touch-first design. *Real-time:* no rich application ecosystem and not needed for the 100 ms interactive time scale.
E3 Doctor workstation	**Desktop (personal, time-sharing)** OS, centrally managed	*Handheld:* too small for multiple heavy applications. *Mainframe terminal:* expensive and inflexible. *Real-time:* feature-poor.
E4 Database server	**Clustered** server OS (UNIX-like, with fail-over)	*Single machine:* single point of failure. *Mainframe:* possible but costly and proprietary. *Desktop OS:* not designed for continuous service.
E5 Imaging server	**Symmetric multiprocessor** server OS	*Asymmetric multiprocessing:* the master CPU becomes a bottleneck. *Single-CPU:* cannot process the images quickly enough.

Environment	Chosen OS type	Rejected alternatives and why
E6 Partner link	**Distributed** system (gateway servers speaking standard secure protocols)	*Shared drives or e-mail attachments:* no authentication, no audit, no atomic updates. *Fully merged single system:* partners would lose administrative independence.

3.3 Symmetric versus asymmetric multiprocessing for the imaging server

Asymmetric multiprocessing (AMP): one *master* processor runs the OS and assigns work to *slave* processors. It is simple, and the slaves never contend for kernel data structures. But every system service must pass through the master, so the master becomes a **bottleneck**, and if the master fails the whole system fails.

Symmetric multiprocessing (SMP): every processor runs the same OS image and may execute kernel code. All processors share main memory and can take interrupts. Any idle processor can pick up any ready thread, so the load balances itself. Failure of one processor reduces capacity but does not stop the system.

Criterion	AMP	SMP
Load balancing	Manual / master-decided	Automatic, per-CPU run queues with migration
Kernel bottleneck	Yes (master)	Needs fine-grained locks; mostly solved in modern kernels
Failure of one CPU	Master failure is fatal	Capacity falls, service continues
Complexity	Low	High (locking, cache coherence)
Suitability for MRI processing	Poor	**Good**

Decision for E5: SMP. MRI reconstruction is *data-parallel*: each slice or tile can be processed by a different core. SMP lets the OS spread threads across all cores with no master bottleneck. The caveat, which Task 9 uses, is that the image data should be moved by **DMA** so that CPU cores spend time computing, not copying.

3.4 Clustered system with fail-over for the database server

In an active–passive (or active–active) cluster, each node runs the same database software. Nodes exchange a heartbeat message every second or less. When a node stops responding, the cluster manager (a) *fences* it to make sure it really is down (so it cannot corrupt shared data), (b) starts the database service on the surviving node, (c) takes over the service's virtual IP address and (d) lets clients reconnect. Typical fail-over time is a few seconds to tens of seconds. The data must be on shared storage (a storage-area network) or synchronously replicated between nodes, so that no committed transaction is lost.

A cluster also fixes the "database may stop during updates" fear from the clinical staff: with rolling upgrades, an administrator patches the passive node, fails over to it (a few seconds), then patches the first node. Users see at most a brief reconnect, not an outage.

The cluster has its own risks which a good design must address: split-brain (both nodes believe they are active) is prevented by a quorum or a third witness node and by fencing; shared-storage failure is reduced with RAID and replicated storage.

3.5 Alternative concept and complexity analysis – one large multiprocessor instead of a cluster

If the database server used a single large multiprocessor machine instead of a cluster of smaller machines with fail-over, what would the solution be?

(a) Availability

Availability A = MTBF / (MTBF + MTTR), where MTBF is the mean time between failures and MTTR the mean time to repair. Assume (as an estimate for illustration) that each server has an MTBF of 10,000 hours and takes 4 hours to repair or replace.

Design	Calculation	Unavailability	Downtime per year
Single large machine	A = 10,000 / 10,004 = 0.9996	0.0004	≈ 3.5 hours from failures, plus planned restarts (assume 2 hours/year for patches) ≈ **5.5 hours** (99.94%)
Two-node cluster, independent failures	Both down together: 0.0004^2 = 1.6×10^{-7} → about 5 seconds/year. Fail-over events: 2 nodes × 8,760/10,000 ≈ 1.75 failures/year × 30 s each ≈ 53 s	–	**≈ 1 minute/year** (99.9998%)

The cluster improves availability by roughly 300 times under these assumptions, and also removes planned downtime because of rolling upgrades. The large single machine remains a *single point of failure*: its motherboard, memory controller, power backplane or the OS itself take everything down. (Large machines have redundant power supplies and memory-failover features, which reduce but do not remove this problem.) The result depends on the independence of failures, so cluster nodes should be on separate power feeds and, ideally, in separate rooms.

(b) Cost

Price rises faster than performance for large shared-memory machines because of special interconnects and chipsets. As an estimate, a 64-core, 2 TB machine commonly costs several times the price of two 16–32-core machines. A cluster, however, adds the cost of shared storage or replication, a witness node and extra licences. Overall: the cluster gives more availability per currency unit, while one large machine can be cheaper to *administer* (one OS image, one set of logs).

(c) Throughput growth with the number of processors (time)

Amdahl's law gives the speed-up with N processors when a fraction p of the work is parallel: $S(N) = 1 / ((1 - p) + p/N)$. For a database with $p = 0.95$ (locks, logging and commit ordering make 5% serial):

N (processors)	1	8	16	32	64	∞
Speed-up S(N)	1.0	5.9	9.1	12.6	15.4	20.0

The curve flattens: going from 16 to 64 cores (four times the cost) gives only 1.7 times the throughput. Real databases scale better if the data is partitioned across several nodes (sharding or read replicas), which is what a cluster or distributed database can do and a single machine cannot do beyond its memory-bus limit.

(d) Memory (space)

One large machine has one large, coherent memory (for example 2 TB), which allows a very large buffer cache and simple programming. In a shared-nothing cluster, each node has less memory, but the total memory across nodes can exceed the single machine and the cache is divided by data partition. Replicated nodes duplicate their caches, so only part of the total memory holds unique data.

Verdict for Task 2

Keep the cluster. The high-availability requirement is ranked first for E4, and a single machine cannot meet it. A strong compromise is *two medium SMP servers in a cluster*: each node uses multiple cores for throughput (Amdahl), and the cluster provides fail-over.

4. Task 3 – Evaluate Operating-System Structures

4.1 The five structures

Simple (monolithic) structure. The whole OS is one large program running in kernel mode: scheduler, memory manager, file systems, network stack and all drivers share one address space and call each other directly as function calls. MS-DOS was simple but had no protection; UNIX and Linux are monolithic kernels with much better organisation. *Advantage:* fastest, since a service is a function call after a single mode switch. *Disadvantage:* a bug in any component, including any driver, can corrupt the whole kernel.

Layered structure. The OS is divided into layers, layer 0 being hardware and layer N the user interface; each layer uses only the services of the layer below. *Advantage:* simple debugging and verification, because each layer can be tested in turn. *Disadvantage:* it is hard to define the layers perfectly, and each request crosses many layers, which costs time. Pure layering is rare today; the *idea* appears in network stacks and as a design discipline.

Microkernel structure. Only the essentials stay in the kernel: address spaces, scheduling, and inter-process communication (IPC). File systems, drivers and network stacks run as ordinary user-mode **servers** that exchange messages. *Advantage:* a failing driver is just a crashed process that can be restarted; the kernel is small and can be formally verified; easy to extend. *Disadvantage:* message passing and extra mode switches cost time.

Modular structure (loadable kernel modules). A monolithic core with a defined interface through which modules (drivers, file systems) are loaded and unloaded at run time. *Advantage:* flexibility and small memory (load only what is needed) with monolithic speed. *Disadvantage:* a loaded module still runs in kernel mode, so a bad module can crash the kernel.

Hybrid structure. A mixture: for example a microkernel-style design in which many services, for performance reasons, run inside the kernel. Most commercial systems are hybrids.

4.2 Effect of each structure on reliability, performance and ease of updating

Structure	Reliability	Performance	Ease of updating
Simple / monolithic	Low–medium: any fault is fatal	Highest: direct calls	Hard: relink or reboot
Layered	Medium: easy to verify	Lower: crossing layers	Medium
Microkernel	High: faults isolated in user processes	Lower per request (IPC), predictable	Easy: restart one server
Modular	Medium	High	Easy: load/unload modules (but kernel faults remain)
Hybrid	Medium–high	High	Medium

4.3 How real operating systems are structured

OS	Structure	Notes
Linux	Monolithic kernel with **loadable kernel modules**	The kernel is monolithic for speed; drivers and file systems are modules. A kernel fault typically causes a "panic" or an "oops". User-space daemons provide many services.
Windows	**Hybrid**	NT kernel plus an executive in kernel mode, a hardware-abstraction layer, and user-mode subsystems; many drivers run in kernel mode, but some run in user mode (user-mode driver framework).
Android	Layered stack on a **modified Linux kernel**	Layers: Linux kernel (with power-management and security additions), hardware-abstraction layer, runtime (ART), framework, applications. Each app runs in its own sandbox/process with its own user ID.
FreeRTOS	Very small kernel library linked with the application (monolithic, single address space, usually no memory protection)	Kernel is a few thousand lines; provides tasks, queues, semaphores, timers. Memory-protection (MPU) variants exist. Ideal for tiny microcontrollers.
QNX	**True microkernel**	The kernel (tens of kilobytes) offers threads, scheduling and message-passing IPC; drivers, file system and network are user-mode processes; fixed-priority pre-emptive scheduling and bounded latencies

4.4 Choice of structure for two environments

Bedside monitor – microkernel real-time OS. The monitor's *worst-case behaviour* and *fault isolation* matter more than raw speed. (i) A fault in the display driver or in a network stack must never stop the alarm task: in a microkernel these run in separate address spaces at lower priority, and the alarm task has the highest priority. (ii) The small kernel is easier to analyse and certify for medical use. (iii) Individual services can be restarted in milliseconds without rebooting. (iv) The extra cost of message passing is tiny compared with the 10 ms budget (see below).

Database server – modular monolithic (Linux-style) kernel. Here throughput is ranked third and reliability is provided mainly by the cluster rather than by the kernel. A monolithic kernel gives the lowest per-I/O overhead (direct file-system and network calls), a huge ecosystem of tested drivers, and modules that can be loaded for new storage hardware without recompiling. A fault on one node is handled by fail-over to the other node; therefore the weaker fault isolation of a monolithic kernel is an acceptable risk.

Why the answers differ. The monitor must be safe *within one machine* because it has no replica; the database can rely on a *second machine* for safety, so it can give up some isolation in exchange for speed.

4.5 Alternative concept and complexity analysis – monolithic kernel on the bedside monitor

If the bedside monitors used a monolithic kernel instead of a microkernel, what would the solution be?

(a) Time cost of a service request

In a monolithic kernel one **system call** is one trap and return. We measured a null system call (syscall(39), getpid) at **106 ns** on the test machine (inside a virtual machine, so this includes virtualisation and security-mitigation overhead). Microkernel services need *two* message transfers (request and reply) and at least two context switches; typical published figures for good microkernels are about **1–5 µs per round trip** (typical value, not measured here).

Request type	Monolithic	Microkernel (assumed 3 µs per IPC round trip)
One service request	≈ 0.1 µs (measured null call)	≈ 3 µs (typical)
Alarm path with 5 service requests	≈ 0.5 µs	≈ 15 µs
Share of the 10 ms deadline	0.005%	0.15%

The microkernel is about **10–30 times slower per request**, but the difference (about 15 µs) is under **0.2%** of the 10 ms budget. Speed is therefore *not* the deciding factor for the monitor; guaranteed behaviour is.

(b) Memory taken by the kernel

A full Linux kernel with the modules needed for a typical device is **tens of MB** (a compressed image of several MB, and 20–40 MB or more in memory once loaded); a QNX-style microkernel is **tens of KB to about 100 KB**, and the whole system including the needed servers is a few MB. For a monitor with 64 MB of RAM this changes the amount left for sensor buffers and trend storage by a considerable fraction, and more code means more bugs (Linux has tens of millions of lines of code; a microkernel has a few tens of thousands).

(c) Effect of a faulty driver on the whole system

Fault	Monolithic	Microkernel
Bug in display driver	Kernel crash (panic) → **all** tasks including the alarm stop until reboot (tens of seconds)	Driver process dies; the microkernel restarts it in milliseconds; the alarm task is unaffected
Memory corruption by network driver	Can overwrite scheduler tables or alarm data	Confined to the driver's own address space
Alarm availability	Depends on the worst driver	Depends only on the small kernel and the alarm task

Drivers are known to be the main source of operating-system crashes (the proportion of kernel bugs in drivers in published studies is typically several times higher than in the core kernel). On a device whose purpose is to alarm, isolating them is worth a small performance cost.

5. Task 4 – Evaluate the Services an Operating System Must Provide

5.1 The services

An operating system offers two groups of services. Services for the user and programs: user interface, program execution, I/O operations, file-system manipulation, communication, and error detection. Services for the efficient running of the system itself: resource allocation, accounting, and protection and security. The table defines each service in hospital terms.

Service	What it does	Hospital example
User interface	Command line, GUI, touch or batch interface	Touch screen on tablet; GUI on workstation; none or fixed display on monitor
Program execution	Loads a program into memory, runs it, ends it (normally or by error)	Starting the record viewer; starting the alarm task at boot

Service	What it does	Hospital example
I/O operations	Programs cannot control devices directly; the OS does it	Sensor read; MRI transfer; printing a wristband
File-system manipulation	Create, delete, read, write, search files and directories; permissions	Patient files; configuration; logs
Communication	Exchange of data between processes on one machine or across a network (shared memory, message passing)	Monitor → central station; hospital → partner
Error detection	Detect and handle CPU, memory, device, and program errors	ECC memory errors; disk failure; sensor disconnect
Resource allocation	Divide CPU, memory, storage and devices among competing jobs	CPU time for imaging threads; memory for queries
Accounting	Track who uses how much	Audit trail of record access; billing of imaging time
Protection and security	Control access to resources; defend against outsiders	Patient privacy; login; encryption

5.2 Service table for each environment

Key: ● essential ◐ reduced / limited form ○ removed or not needed.

Service	E1 Monitor	E2 Tablet	E3 Workstation	E4 DB server	E5 Imaging	E6 Partner link
User interface	◐ fixed alarm display, no shell	● touch GUI	● full GUI	○ remote shell only	○ remote shell only	○ none
Program execution	◐ fixed set started at boot	● app sandbox	● multi-program	● services	● many threads/jobs	● gateway services
I/O operations	● sensors, display, alarm output, network	● touch, camera/barcode, Wi-Fi	● standard	● disk, network	● **DMA** to MRI/storage	● network
File systems	◐ small flat log and configuration	● encrypted storage	● full	● **journaling, large**	● large-file, parallel	◐ spool/queue
Communication	◐ one authenticated channel to central station	● Wi-Fi, VPN	● network	● client and cluster messages	● internal network	● **encrypted** to partners
Error detection	● watchdog, sensor-fault check	●	●	● ECC, RAID, journaling, heartbeats	● ECC, checksums	● retries, integrity checks

Service	E1 Monitor	E2 Tablet	E3 Workstation	E4 DB server	E5 Imaging	E6 Partner link
Resource allocation	◐ static, fixed priorities	● power-aware	● fair sharing	● quotas, cgroups	● CPU/NUMA scheduling	◐
Accounting	○ (local counters only)	◐	◐	**● audit trail**	◐ job accounting	● audit trail
Protection and security	**● memory protection, signed firmware**	● encryption, remote wipe	● accounts, antivirus	● ACLs, mandatory access control	●	**● mutual authentication**

Justification (selected rows).

- **E1 – user interface.** A monitor shows vital signs and alarms; a command shell, login manager or window system adds code, memory, and attack surface. A fixed display application is enough.
- **E1 – accounting and multi-user services.** There are no users to bill; keeping per-user accounting wastes memory and CPU.
- **E4 – accounting.** Hospitals must record who looked at which patient record (privacy law and audit). Accounting is therefore a *safety and compliance* service in E4, not an optional extra.
- **E5 – I/O.** The imaging server moves enormous files, so a first-class DMA path and a file system for large sequential files are essential.
- **E6 – communication.** Mutual authentication and encryption are treated as part of the communication service because the network is untrusted.

5.3 Services that can be removed from the bedside monitor to make it smaller and faster

The following can be removed, with a reason for each:

1. **Window system and GUI toolkit** – replaced by a fixed rendering routine (saves several MB and removes a large attack surface).
2. **Shell, login and multi-user accounts** – nobody logs into a monitor; clinical engineers use a signed maintenance interface.
3. **General-purpose file system and journaling** – a small flat store for configuration and trend data is enough (removes caches and background write-back that cause unpredictable delay).

4. **Virtual memory and swapping** – paging introduces unbounded delay. A real-time design *locks* all alarm code and data in RAM.

Result: a smaller kernel and image (flash and RAM use fall by more than an order of magnitude), shorter boot time (seconds instead of tens of seconds), fewer interrupts and background activity, and a *provable* response time.

5.4 Services that are essential for the database server

1. **A reliable, journaling file system** – so that a crash never leaves a half-written record. Journaling writes the intention to a log first and replays it after a crash.
2. **Efficient I/O with caching** – the buffer cache makes repeated record look-ups memory-speed (Task 9).
3. **Process and thread management with fair scheduling and resource limits** – so that one runaway query cannot starve other users.
4. **Error detection and recovery** – ECC memory, RAID, checksums, watchdogs, and cluster heartbeat messages.
5. **Communication** – network stack for clients and a private link between cluster nodes.
6. **Protection, security and accounting** – authentication, access control lists, audit trail, encryption in transit and at rest.
7. **Updating without outage** – live patching, or rolling upgrades with the cluster.

5.5 How the OS protects one patient's records from another user's programs

Protection is built in layers, each of which defends against a different attack. The layers use mechanisms examined in Task 8.

Layer	Mechanism	What it stops
1. Hardware memory protection	Each process has its own virtual address space; the MMU translates addresses and refuses accesses outside it (a trap occurs)	A program reading another program's memory
2. CPU modes	Only kernel mode can execute privileged instructions and access devices	A program talking directly to the disk
3. Authentication	Each user and service has a unique identity (password, token, certificate)	Anonymous access
4. File and object permissions	Owner/group/other bits and access control lists on each file or table	One user opening another department's files
5. Mandatory access control (e.g. SELinux-style labels)	System-wide policy that even the owner cannot override	A compromised service reading records it should not touch

Layer	Mechanism	What it stops
6. Application-level controls	Database roles and row-level security: a clinician sees only the patients assigned to them, with "break-glass" emergency access that is logged	Legitimate users looking at records they have no care reason to read
7. Encryption	Records encrypted on disk and on the network	Theft of disks or network eavesdropping
8. Audit	Accounting service logs every access	Undetected misuse; provides evidence

When a program asks to read a record, the **system call** enters the kernel; the kernel checks the caller's identity against the file's permissions *before* any data is returned; a failed check returns an error , not the data. The hardware makes the kernel the only path to the disk.

5.6 Alternative concept and complexity analysis – tablets with a full desktop OS

If the tablets used a full desktop operating system with every service enabled instead of a reduced handheld OS, what would the solution be?

(a) Space: memory and storage used by the extra services

Resource	Reduced handheld OS	Full desktop OS, all services	Difference
Storage for OS image	4–8 GB	20–30 GB	about 15–25 GB extra per tablet
RAM used at idle	1–1.5 GB	3–4 GB	about 2–3 GB extra
Background processes	tens	100 or more	more scheduler load

These are **typical estimates**. A tablet with 4 GB of RAM running a desktop OS would be left with almost nothing for the clinical app, causing paging to flash storage, which both slows the UI and wears the flash. Across 120 tablets, the extra storage is about 2–3 TB of flash, bought to hold software the nurse never uses.

(b) Time: start-up and battery life

Measure	Reduced handheld OS	Full desktop OS	Notes
Cold boot	10–20 s	30–60 s or more	Many services start
Resume from sleep	< 1 s	2–5 s	Desktop services must wake
Average power draw (estimate)	≈ 2.5 W	≈ 5 W	Background services, no aggressive suspend
Battery life on a 30 Wh battery	30 / 2.5 = **12 h**	30 / 5 = **6 h**	Runs out in the middle of the shift

Battery life is calculated as *energy ÷ power*. With the example figures, the desktop OS cuts a 12-hour shift to about 6 hours. Mobile operating systems save power with aggressive suspend, restricted background work, app lifecycle management, and wakelocks that the OS can inspect and penalise; desktop systems assume mains power.

Verdict for Task 4

Use a reduced **handheld OS**. A desktop OS would halve the battery life (the first-ranked requirement for E2), consume large amounts of storage, and enlarge the attack surface on a device that can be lost.

6. Task 5 – Examine System Calls (4 marks)

6.1 What a system call is

A system call is the programmatic interface between a running program and the operating system kernel. A user program runs in user mode and cannot touch devices or other processes' memory; to do anything involving those it must *ask the kernel*, which runs in kernel mode and acts on its behalf after checking permission. Programmers rarely use system calls directly; they call library functions (for example in the C library) that wrap the system calls, or use an API such as POSIX or Win32.

6.2 Categories of system calls

Category	Purpose	Examples (UNIX/Linux)	Hospital example
Process control	Create, run, wait, end processes; allocate memory	fork, execve, wait, waitpid, exit, brk, mmap, kill	Shell starts a backup job
File management	Create, open, read, write, close, delete files; get/set attributes	open, read, write, close, lseek, unlink, stat	Reading a patient file
Device management	Request/release devices; read/write; attach/detach	ioctl, read, write on device files, mmap of device memory	Reading a sensor; controlling the MRI
Information maintenance	Get/set time, system data, process attributes	time, gettimeofday, getpid, uname, sysinfo	Time-stamping a reading
Communication	Create/delete connections; send and receive messages; shared memory	pipe, socket, connect, send, recv, shmget, mq_send	Monitor sends alarm to nurses' station
Protection	Control access to resources; set permissions; users and groups	chmod, chown, umask, setuid	Setting record permissions

6.3 What happens when a program calls `read()` – step by step

Suppose the program contains `n = read(fd, buf, 4096);` on x86-64 Linux.

Step	What happens	Mode
1. **Library call**	The program calls the C-library wrapper read(). This is an ordinary function call.	User
2. **Load parameters**	The wrapper puts the system-call number (0 for read on x86-64) in register rax, and fd, buf, count in rdi, rsi, rdx.	User
3. **Trap**	The wrapper executes the syscall instruction (a software trap). The hardware saves the user program counter and flags, loads the kernel entry point, and **sets the mode bit to kernel mode**.	Switch to kernel
4. **Kernel entry**	The kernel switches to the process's kernel stack and saves the remaining user registers.	Kernel
5. **Dispatch**	The kernel indexes the **system-call table** with the number in rax and calls the handler (ksys_read).	Kernel

Step	What happens	Mode
6. **Check parameters**	It validates the file descriptor, checks that buf is a valid writable user address and that the file was opened for reading (permission check).	Kernel
7. **Kernel work**	The virtual file system calls the file system. If the data is in the **page cache**, it is copied to the user buffer. If not, a disk request is issued, the process is put to **sleep**, another process runs, and the disk **interrupt** later wakes it.	Kernel
8. **Return value**	The handler leaves the number of bytes read (or a negative error code) in rax.	Kernel
9. **Return**	The kernel restores registers and executes sysret; the hardware resets the mode bit to user mode and resumes the program.	Switch to user
10. **Library return**	The wrapper checks rax; if it is a negative error number, it stores it in errno and returns −1; otherwise it returns the count.	User

The program then continues after the call, unaware of all the steps in between. The expensive parts are the privilege switch, the saving and restoring of registers, and, if data is not cached, the wait for the device.

6.4 How parameters are passed to the kernel

Method	How it works	Used by	Advantages and disadvantages
Registers	Parameters are placed in CPU registers	Linux x86-64: rdi, rsi, rdx, r10, r8, r9; number in rax	Fastest, no memory access; limited to the number of registers (at most 6 here)
Block (table) in memory	Parameters are stored in a block in memory and the **address of the block** is passed in a register	Linux when there are more parameters than registers (older ABIs and calls such as mmap on 32-bit x86)	Any number and size of parameters; one memory access more
Stack	The program **pushes** parameters on the stack and the OS pops them	Some other systems and older ABIs	No limit on the count; needs care to validate user addresses

In all three, the kernel must **validate** every pointer because it cannot trust the program. Block and stack methods do not limit the number or length of parameters; the register method is the fastest.

6.5 Using strace to see the system calls of a program

strace uses the ptrace facility to stop a program at each system-call entry and exit and print the name, arguments and return value. Useful options:

Command	Meaning
strace ./copy	Print every system call with arguments and results
strace -c ./copy	Count calls and time per system call and print a summary table at exit
strace -e trace=read,write ./copy	Show only the listed calls
strace -T ./copy	Show time spent in each call

Command	Meaning
strace -o out.txt ./copy	Save the trace to a file
strace -f ./shell	Follow child processes after fork (needed in Task 7)

A typical line looks like read(3, "ABCDEFGHIJKLMNOPQRSTUVWXYZABCDEF"..., 4096) = 4096: the call, its arguments, and after = the returned value, which also confirms steps 8–10 above. Failed calls show the error, for example openat(AT_FDCWD, "x", O_RDONLY) = -1 ENOENT (No such file or directory).

Evidence note. strace was not installed on the test machine, so the program of Task 6 was instrumented with its own counters instead (Section 7.4). To produce the strace evidence required by the task, run on any Linux machine: gcc -O2 -o copy copy.c && strace -c ./copy. Insert your strace output here. Section 7.5 states what it should show, derived from the program's counters.

6.6 Alternative concept and complexity analysis – fopen/fread/fwrite instead of open/read/write

If programs called the C-library functions fopen, fread and fwrite instead of the open, read and write system calls directly, what would the solution be?

The stdio functions keep a **buffer in user space** (the size is usually the file system's block size, 4,096 bytes on most Linux file systems, and 8,192 on others). fread(buf,1,1,f) copies one byte from the library buffer without any system call, and calls read() only when the buffer is empty. fwrite accumulates data and calls write() only when its buffer is full (or on fflush/fclose).

(a) Number of system calls for a file of n bytes

Method	Read-side calls (about)	Example, n = 10,000 bytes	Example, n = 16 MiB = 16,777,216 bytes
Direct read() of b bytes each time	n / b	b = 1: **10,000** (plus 1 for end-of-file)	b = 1: **16,777,216**
Library fread with 4,096-byte buffer, even if the program asks for 1 byte at a time	n / 4096	**3** (plus 1)	**4,096**
Direct read() with b = 4,096	n / 4096	3 (plus 1)	4,096

We measured the direct-read counts for n = 10,000 in Task 6: 10,001 calls for b = 1 (10,000 data reads plus a final read that returns 0 for end-of-file), 21 for b = 512 and 4 for b = 4,096. The values n/b are rounded up and one more call detects the end of file. Library buffering therefore gives a **speed-up factor of about 4,096 in calls** for a program that processes the file byte by byte, without the programmer having to manage a buffer.

(b) Extra buffer space

Each open FILE has its own buffer: **about 4 KiB** (plus a FILE structure of a few hundred bytes). A program holding 1,000 open files uses roughly 4 MB for library buffers, and for a copy program the data is copied *twice* (kernel → library buffer → user variable) instead of once. For the hospital this overhead is negligible on servers and workstations, but on a monitor, where memory is small and the code must be deterministic, the programmer should use read/write directly with a fixed pre-allocated buffer.

7. Task 6 – Evaluate the Cost of System Calls (4 marks)

7.1 Why a system call costs far more than an ordinary function call

An ordinary function call is a jump, a few register moves and a return: roughly 1–2 nanoseconds (typical value). A system call does all of that plus the following extra work:

1. **Mode switch** – the `syscall` instruction changes the privilege level; the pipeline is serialised.
2. **Saving and restoring state** – user registers are saved on the kernel stack and restored on return.
3. **Security mitigations** – on many CPUs, entering the kernel may switch page tables or flush predictors to defend against speculative-execution attacks.
4. **Validation** – the kernel checks the call number, the file descriptor, pointers and permissions.
5. **Cache and TLB disturbance** – kernel code and data evict user lines from the cache.
6. **Possible scheduling** – on return the kernel may decide to run another process, and if the data is not ready the process sleeps and wakes later.

Measurement. A tight loop calling the cheapest system call (`syscall(39)`, getpid) 5,000,000 times took **106.3 ns per call** on the test machine; a trivial arithmetic loop step took **0.34 ns**. Compared with a normal function call of about 1–2 ns, a system call is therefore roughly **50–100 times more expensive** even when it does *no* work. A `read()` that copies data costs more again, because of the copy and the file-system work. (Measured inside a virtual machine; on bare metal the numbers are often smaller but the ratio remains of the same order.)

7.2 How buffer size changes the number of `read()` and `write()` calls

If the program copies a file of n bytes with a buffer of b bytes, the loop executes $\lceil n / b \rceil$ times with data, and one more `read()` that returns 0 to signal the end of file. Each loop iteration makes one `read()` and one `write()`. The number of `read()` calls is therefore $\lceil n/b \rceil + 1$ and the total number of system calls is **about $2 \times (\lceil n/b \rceil + 1)$**, plus a few for open and close.

Buffer b	n = 10,000 bytes	n = 16 MiB (16,777,216 bytes)
1 byte	10,001	16,777,217
512 bytes	21	32,769
4,096 bytes	4	4,097
65,536 bytes	1 + 1 = 2	257
1 MiB	2	17

The formula reproduces the figures expected in the task (10,001, 21, and 4) for the 10,000-byte test file.

7.3 How library buffering reduces the number of system calls

The standard library collects many small `fwrite` or `fread` requests into one 4 KiB block (or 8 KiB) and issues a single `write` or `read` for the whole block. A program that does `fputc` 10,000 times therefore makes only **3 write calls** (4,096 + 4,096 + 1,808 bytes) when the stream is closed instead of 10,000. The test program uses precisely this when it creates `records.dat` with `fputc`, which is why creating the file is cheap while copying it one byte at a time is expensive. (See Task 5, Section 6.6, for the formula.)

7.4 Completed code

The task's skeleton declares `char buf[8192]` while `bufsize` is a parameter. The test calls use at most 4,096, but if somebody called the function with a larger size, the `read()` would overflow

the array, so the completed code **clamps** bufsize to the size of the array. This small defensive step is part of correct system-call programming: the kernel will write as many bytes as it is asked to.

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
/* Copy src to dst with read()/write() using a buffer of bufsize bytes.
   Returns the number of read() calls made, or -1 on error. */
long copy_file(const char *src, const char *dst, size_t bufsize)
{
    char buf[8192];
    int in = open(src, O_RDONLY);
    if (in < 0) return -1;
    int out = open(dst, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (out < 0) { close(in); return -1; }
    long calls = 0;
    ssize_t n;

    if (bufsize > sizeof buf) bufsize = sizeof buf;   /* never overflow buf */

    /* TODO 1 (completed) */
    while (1) {
        n = read(in, buf, bufsize);   /* one read() system call            */
        calls++;                      /* count every call, including the   */
                                      /* last one that returns 0 (EOF)     */
        if (n <= 0) break;            /* 0 = end of file, -1 = error       */
        if (write(out, buf, n) != n) {/* write exactly the n bytes read    */
            calls = -1;               /* short write or error              */
            break;
        }
    }
    close(in);
    close(out);
    return calls;
}

int main(void)
{
    FILE *f = fopen("records.dat", "w");   /* a 10,000-byte test file */
    for (int i = 0; i < 10000; i++) fputc('A' + i % 26, f);
    fclose(f);
    size_t sizes[] = {1, 512, 4096};
    for (int i = 0; i < 3; i++)
        printf("buffer %4zu bytes: %5ld read() calls\n", sizes[i],
               copy_file("records.dat", "copy.dat", sizes[i]));
    unlink("records.dat");
    unlink("copy.dat");
    return 0;
}
```

Design notes. (1) read may legally return fewer bytes than requested (for example at the end of the file, or from a pipe), so the loop writes exactly n bytes, not bufsize. (2) A negative return from read or a short write is treated as an error; a production backup program would also retry when errno == EINTR. (3) calls++ comes straight after read, so the final call that returns 0 is counted, which is what produces 10,001, not 10,000.

Actual output of the program on Linux (compiled with gcc -O2):

```
buffer    1 bytes: 10001 read() calls
buffer  512 bytes:    21 read() calls
buffer 4096 bytes:     4 read() calls
```

This matches the expected output exactly.

7.5 Comparing with strace -c

The task asks to run strace -c ./copy and compare the read() count with the program's output. strace was not installed on the test machine (no package installation was possible), so this comparison could not be executed here. The expected result, **derived from the program's own counters** and clearly not a copy of real strace output, is:

Item	Value predicted from the counters	Why
read calls in the table	**about 10,027**	10,001 + 21 + 4 = 10,026 from copy_file, plus 1 by the dynamic loader reading the C library header
write calls	**about 10,027–10,029**	10,000 + 20 + 3 = 10,023 data writes in copy_file, 3 writes to create records.dat, and 1–3 for printf output
openat calls	about 8–10	3 copy_file × 2 + fopen + loader files

An independent test that does **not** need strace: the three numbers printed by the program are the real counts of calls that reached the kernel, because each read() in the loop is exactly one system call. When you run strace -c ./copy yourself, check that the calls column for read is about 10,027, i.e. the sum of the three program lines plus one or two start-up reads. **Insert your own strace output here.**

7.6 Measurements and the best buffer size for the backup program

A larger benchmark copied files with a heap buffer of each size. The 1-byte and 16-byte rows use a 1 MiB file (to keep the run short; the 1-byte row's call count is for 1 MiB), and the other rows copy a 16 MiB file. Times were taken with clock_gettime and getrusage; each row is one run on a one-CPU virtual machine, with the files in the page cache.

Buffer b	File size	read() calls	Real time (s)	User CPU (s)	System CPU (s)
1 byte	1 MiB	1,048,576	0.4928	0.1194	0.3718
16 bytes	16 MiB	1,048,576	0.5276	0.1184	0.4049
64 bytes	16 MiB	262,144	0.1364	0.0282	0.1005
512 bytes	16 MiB	32,768	0.0423	0.0009	0.0231
4,096 bytes	16 MiB	4,096	0.0193	0.0005	0.0097
65,536 bytes	16 MiB	256	0.0299	0.0003	0.0090
1,048,576 bytes	16 MiB	16	0.0126	0.0004	0.0053

Reading the table.

- The call counts follow n/b exactly (16 MiB / 16 B = 1,048,576; 16 MiB / 4 KiB = 4,096; 16 MiB / 1 MiB = 16).
- For 1-byte copying the cost is **about 0.49 s per 1 MiB** (two system calls per byte, about 235 ns per call), and **75% of the time is system time**, i.e. kernel work, not user work. Extrapolated linearly to 16 MiB this is about **7.9 s**, compared with 0.0193 s at 4 KiB, i.e. **roughly 400 times slower**, and about 600 times slower than a 1 MiB buffer (0.0126 s).

- Between 1 byte and 512 bytes, doubling the buffer almost halves the time. After about 4 KiB, the curve is flat: the remaining time is the kernel copying data and moving it through the file system, which no buffer size can remove.
- **The 64 KiB row (0.0299 s) is slower than the 4 KiB row (0.0193 s).** Theory says it should not be. With one run per row on a virtual machine, such differences are within the noise (page-cache state, scheduler interruptions, CPU frequency). A careful experiment should repeat every row at least five times and report the **median**. We therefore *do not conclude* that 64 KiB is worse; we conclude that sizes from 4 KiB up are in the same flat region.

Overhead check. At 4 KiB the 8,192 system calls account for about 8,192 × 0.235 µs ≈ **1.9 ms** of the 19.3 ms total (10%). At 64 KiB they account for about 512 × 0.235 µs ≈ **0.12 ms**, which is negligible.
Recommendation for the hospital's backup program. Use a buffer of **64 KiB** (a multiple of the 4 KiB page and block size). Reasons: (1) it reduces system calls by a factor of 16 compared with 4 KiB and by a factor of 65,536 compared with 1 byte; (2) it is small enough to stay in the CPU cache and costs only 64 KiB of memory per copy stream; (3) it fits the read-ahead window of the file system (usually 128 KiB), so the disk sees large sequential requests; (4) beyond this, our measurements show no reliable gain, but memory use and the time to *notice* an error grow. For backup to a network or tape device, larger buffers (256 KiB–1 MiB) may help because each request has a much larger fixed latency, and this should be measured on the real target.

7.7 Alternative concept and complexity analysis – memory-mapped files (`mmap`)

If the backup program used memory-mapped files (mmap) instead of read() and write() with a buffer, what would the solution be?
With `mmap` the program asks the kernel to map the file into its address space. After that the file is just an array: reading `src[i]` makes the CPU access memory; if the page is not yet in memory a **page fault** occurs and the kernel loads it.

Aspect	read/write with buffer b	mmap
System calls	about 2n/b (one pair per buffer)	**a few**, independent of n: open×2, ftruncate, mmap×2, msync, munmap×2, close×2 (about 10)
Copying between kernel and user space	Data is copied from the page cache into the user buffer, then back into the kernel for write (two copies)	Mapped pages *are* the page cache: no kernel→user copy on the read side; the program copies directly between the two mappings
Memory for large files	Constant: b bytes, whatever the file size	Virtual address space of the size of the mapping; physical memory is used page by page and clean pages can be evicted, but a 64 GB file cannot be mapped whole on a 32-bit system, so a large file must be mapped in windows
Costs that are not obvious	System-call cost, 2 per buffer	**One page fault per 4 KiB page** the first time it is touched, plus msync to force data to disk

Measured. Copying a 16 MiB file with mmap (two mappings, a 4 KiB memcpy loop, then msync) took **0.0378 s**, which is **slower** than read/write with a 4 KiB buffer (0.0193 s) and

slower than a 1 MiB buffer (0.0126 s). The system-call count is far lower, but 4,096 pages × 2 mappings means about 8,192 page faults, each costing about as much as a system call, and msync(MS_SYNC) waits for the disk.
Conclusion. mmap removes system calls but does not remove cost. It wins when a file is accessed **randomly** or **repeatedly** (a database index), or shared between processes. For a **one-pass sequential backup**, read/write with a 64 KiB buffer is as fast or faster, simpler, and has constant memory use. This supports the recommendation above.

8. Task 7 – Evaluate Process Control System Calls (4 marks)

8.1 What fork(), exec(), wait() and exit() each do

Call	Action	Return value
fork()	Creates a **new process** that is a copy of the caller (the *parent*): same code, same data (copy-on-write), same open files.	Returns the **child's PID** to the parent and **0** to the child; −1 on error.
exec() family (execlp, execve, …)	**Replaces** the program in the calling process with another program: code, data and stack are replaced; the process ID and open files remain.	Does not return on success; −1 if it fails (for example, command not found).
wait() / waitpid()	The parent **blocks** until a child terminates and then collects its **exit status**.	The PID of the finished child; the status is stored via a pointer.
exit() / _exit()	Ends the process, releases memory and files, and passes an exit status (0 = success) to the parent. exit also runs atexit handlers and **flushes stdio buffers**; _exit does neither.	Does not return.

8.2 How a shell runs a command and gets the exit status

When the user types expr 6 * 7:

1. The shell reads and parses the line (command name expr, arguments 6, *, 7).
2. The shell calls fork(). Now two almost identical processes exist.
3. In the **child**, the shell sets up redirection and pipes if needed, then calls execlp("expr", "expr", "6", "*", "7", NULL). The kernel searches the PATH, loads expr and starts it. If the command does not exist, exec returns, and the child calls exit(127) (the conventional code for "command not found").
4. The **parent** calls waitpid(pid, &status, 0) and sleeps.
5. expr writes 42 to standard output and exits with status 0.
6. The kernel wakes the parent; waitpid returns and fills in status. The macro WEXITSTATUS(status) extracts the 8-bit exit code, which the shell stores in $?.

8.3 Zombie and orphan processes

- A **zombie** is a process that has **finished** (called exit) but whose parent has not yet called wait. The kernel must keep a small record (PID, exit status, resource usage) so that the parent can still collect the status. It uses no CPU or memory except that table entry, but it occupies a **process-table slot**; thousands of zombies can prevent new processes from being created. *Remedy:* the parent calls wait/waitpid (or handles SIGCHLD, or ignores it explicitly); if the parent itself dies, the zombie is inherited by the init process, which reaps it.

- An **orphan** is a process that is **still running** after its parent has ended. The kernel re-parents it to the init process (PID 1, or a designated "subreaper"), which later calls wait when it finishes, so orphans do not become permanent zombies.

For the hospital's database server a long-running supervisor that starts jobs must always reap its children; a leak of zombies would eventually exhaust the process table.

8.4 Why fork() followed by exec() and not a single "create process" call

The two-step design lets the child **modify its own environment between fork and exec** using ordinary system calls, without a vast parameter list: redirect standard input and output (dup2), close files, change the working directory, set environment variables, change user ID, set resource limits or put itself in a new process group. This is how a shell implements >, <, | and &. A single CreateProcess-style call (as in Windows) needs a big structure with parameters for every such option. Disadvantages of fork: copying the page tables of a big parent costs time (reduced by copy-on-write and by vfork/posix_spawn), and the interface is awkward with threads.

8.5 Completed code

The skeleton declares pid and status only in part of the code; the completed program declares them at the top and fills the missing block. The fflush before fork is explained in Section 8.6.

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
   int status;
   pid_t pid;

   printf("shell: running job 1\n");
   fflush(stdout);              /* empty the buffer before fork() */

   /* TODO 1 (completed) */
   pid = fork();
   if (pid < 0) { perror("fork"); return 1; }
   if (pid == 0) {                          /* child */
      execlp("expr", "expr", "6", "*", "7", (char *)NULL);
      exit(127);                            /* reached only if exec fails */
   }
   waitpid(pid, &status, 0);                    /* parent waits */
   printf("job 1 exit status %d\n", WEXITSTATUS(status));

   printf("shell: running job 2\n");
   fflush(stdout);
   pid = fork();
   if (pid == 0) _exit(3);       /* a child that fails with code 3 */
   waitpid(pid, &status, 0);
   printf("job 2 exit status %d\n", WEXITSTATUS(status));
   return 0;
}
```

Actual output on Linux:

```
shell: running job 1
42
job 1 exit status 0
```

```
shell: running job 2
job 2 exit status 3
```
Identical to the expected output. The status values come from WEXITSTATUS: 0 from expr (it succeeded; the expression result was non-zero, which would give status 0 in expr) and 3 from the child's own _exit(3).

8.6 Why stdout is flushed before each fork(), and the duplicate-output experiment

Why flush. The printf function writes into a user-space buffer. When stdout is a **terminal** the buffer is line-buffered and flushed at each newline, but when stdout is a **file or pipe** it is *fully buffered* and flushed only when full, on fflush, or at normal exit. fork() copies the whole process memory, including that buffer. If unflushed text is in the buffer at the time of fork, **both** the parent and the child hold a copy and each will write it eventually, so the text appears twice. Flushing first makes the buffer empty at the fork so that nothing is duplicated.
The experiment. The second fflush(stdout) was removed, _exit(3) was changed to exit(3), and the output was redirected to a file (./shell2 > out.txt). The real content of out.txt was:
```
shell: running job 1
42
job 1 exit status 0
shell: running job 2
job 1 exit status 0
shell: running job 2
job 2 exit status 3
```
The line **"shell: running job 2" now appears twice, and so does "job 1 exit status 0"** (it was in the same unflushed buffer). Explanation, in order:

1. The first fflush emptied the buffer before job 1's fork, so job 1 is clean.
2. After job 1 the parent printed job 1 exit status 0 and shell: running job 2 into the buffer. Because output goes to a file, the buffer is not flushed at the newline, and the removed fflush no longer flushes it.
3. fork() for job 2 duplicates the buffer, so the child also holds both lines.
4. The child calls exit(3), which **flushes stdio buffers**, so the child writes the two lines to the file (this is the first copy).
5. The parent resumes, prints job 2 exit status 3, and at its own exit flushes the buffer containing the two lines *and* the new line (second copy of the two lines).

Both changes are needed: with the original _exit(3) the child would end *without* flushing its copy of the buffer, so no duplicate would appear even without the fflush. On a terminal there is also no duplicate, because newline flushes a line-buffered stream. The lesson for the hospital: **flush before fork, and use _exit in a child that has not called exec**, otherwise log files could contain duplicated entries, which in a clinical audit trail would be misleading.

8.7 Alternative concept and complexity analysis – posix_spawn() or threads instead of fork + exec

If commands were run with posix_spawn() or with threads instead of fork() followed by exec(), what would the solution be?

(a) Time to start 1,000 commands – measured

Method (1,000 repetitions)	Total time	Per start	Notes
fork() + _exit + waitpid (no exec)	0.0531 s	53 µs	Small parent; copies page tables

Method (1,000 repetitions)	Total time	Per start	Notes
vfork() + _exit + waitpid	0.0086 s	8.6 µs	Child borrows the parent's memory until exit/exec
Thread create + join	0.0086 s	8.6 µs	Shares all memory
fork() + exec("/bin/true") + waitpid	0.7024 s	702 µs	The realistic cost of starting a real command: loading and linking a program dominates

posix_spawn was not measured separately. On Linux the C library implements it with a clone that shares memory until the exec, so its cost should be close to vfork + exec, i.e. at or below the fork+exec row; it is the better choice when the parent is large. This is **expected, not measured**.

Starting a *real command* costs 702 µs, **13 times more than the fork alone**, so for 1,000 commands, running exec (loading the program) is what takes the time, not fork; threads and vfork save only the small part.

(b) Memory copied or shared (copy-on-write)

- **fork:** the child gets copies of the *page tables* (about 8 bytes per 4 KiB page, i.e. about 2 MiB per GiB of parent memory), and all pages are marked read-only and **shared**. A page is physically copied only when someone writes to it. A 10 GB database parent therefore costs roughly 100 ms to fork (extrapolated from the Task 10 measurement of about 9.5 µs per MiB), not seconds, but a child that writes a lot will copy a lot.
- **posix_spawn/vfork:** nothing is copied: the child runs in the parent's address space until it execs. Cheapest for large parents.
- **Threads:** everything is shared (code, heap, global variables, open files); only the stack and registers are private.

(c) Isolation between commands

Method	Isolation
fork + exec / posix_spawn	**Strong**: separate address spaces; a crash or memory error in one command cannot corrupt the shell or other commands; each can have its own user ID and limits
Threads	**None** between threads: one wild pointer or exit() in a command thread kills the whole shell and every other command, and all share the same file descriptors and permissions

9. Task 8 – Evaluate CPU States and Protection

9.1 User mode and kernel mode

The processor has at least two **modes of operation**, recorded in a hardware **mode bit** (or privilege-level field) in a status register.

	User mode (mode bit = 1 in the textbook convention)	**Kernel (supervisor) mode** (mode bit = 0)
Who runs	Applications, libraries, and in a microkernel most drivers and servers	The kernel and, in a monolithic kernel, drivers
Instructions allowed	Ordinary instructions only	**All**, including privileged ones
Memory access	Only the process's own address space	All memory, including other processes
Device access	None directly (through system calls)	Direct

Privileged instructions can be executed only in kernel mode. Examples: instructions that perform I/O directly, load the memory-management registers (page-table base), enable/disable interrupts, set the timer, change the mode bit, and halt the processor. If a user program attempts one, the hardware does not execute it: it raises a **trap** (an illegal-instruction or general-protection exception), and the kernel normally terminates the program.
The mode changes in a controlled way: the CPU enters kernel mode only by a **system call (trap), an interrupt or an exception**, and each of these jumps to a fixed address chosen by the kernel. A user program cannot choose where it goes in kernel mode, which prevents it from taking over the kernel. On return, the kernel sets the mode bit back to user mode before resuming the program.

9.2 Interrupts, traps and the timer interrupt

Event	Cause	Synchronous?	Example	Handled by
Interrupt	A hardware device signals the CPU	Asynchronous (at any time)	Sensor data ready; disk transfer finished; network packet arrived	Interrupt service routine (ISR) found through the **interrupt vector** table
Trap / system call	A software instruction	Synchronous	`syscall` instruction	System-call handler
Exception (fault)	An error during an instruction	Synchronous	Division by zero, illegal memory access, page fault	Exception handler

On an interrupt the CPU finishes the current instruction, saves the program counter and flags, switches to kernel mode, looks up the address of the ISR in the vector table, and runs it; after that the interrupted program resumes (or another is chosen by the scheduler). Interrupts have **priorities**, so that an urgent interrupt can pre-empt a less urgent ISR.
The **timer interrupt** is the hardware that prevents one program from keeping the CPU forever. Before handing the CPU to a user program, the kernel loads a timer with the **time slice** (for example 4 ms). When the timer reaches zero it interrupts the CPU, control passes to the kernel, and the scheduler can choose another process. In a real-time OS, the same timer provides the **tick** that drives periodic tasks such as sampling the sensors.

9.3 What would happen to the monitors if any program could disable interrupts

Suppose any program, including a display or network program, could execute the "disable interrupts" instruction (`cli` on x86).

1. **The timer stops.** No further timer interrupts means the scheduler never regains control; the program that disabled interrupts monopolises the CPU, even if it is in an infinite loop. The alarm task, which is waiting to run, never runs.
2. **Sensor interrupts are lost or delayed.** The abnormal reading arrives but the CPU does not respond. Because many devices keep only one or a few samples in a register, the next sample *overwrites* the abnormal one: a clinically important reading disappears.
3. **Network and storage interrupts are lost.** Messages to the nurses' station are not sent, and the device appears "dead" to the central system.

4. **A bug becomes a safety failure.** A programming error or malicious code in a non-critical component (for example the screen saver) would turn into a missed alarm. Only a watchdog reset could recover, after perhaps seconds.

Because of these consequences, disabling interrupts is a **privileged instruction**: user-mode code gets a trap instead of the effect. Even in kernel mode, drivers in a real-time OS keep interrupt-disabled sections to a few microseconds, and the designers measure and bound them, since this time adds directly to the worst-case alarm latency. A **hardware watchdog timer** adds a second line of defence: the alarm task must "kick" the watchdog every few milliseconds, and if it stops, the hardware resets the device and sounds a local alarm.

9.4 How the hardware stops one program from reading another program's memory

The CPU's **memory-management unit (MMU)** checks every memory access made in user mode.

- **Base and limit registers (simple scheme).** Each process is given a base address and a limit. Every address generated is compared with both; an address outside the range causes a trap to the kernel. Only privileged instructions can change these registers.
- **Paging (modern scheme).** Each process has its own **page table**, set by the kernel, which maps its *virtual* addresses to *physical* frames. Another process's memory is simply not in the table, so it cannot even be named. Each page-table entry also carries permission bits (read, write, execute, user/supervisor); an attempt to write a read-only page, execute a data page (the NX bit) or touch a kernel page from user mode causes a **page fault** or protection fault. The kernel then ends the program (for example, "segmentation fault").
- **Translation lookaside buffer (TLB).** A cache of recent translations keeps this checking fast (typically one clock cycle on a hit).
- **Memory-protection unit (MPU).** Small microcontrollers used in simple monitors have no full MMU, but an MPU provides a handful of protected regions (for example 8 or 16) with access permissions; this is enough to separate the alarm task, the display task and the network task.

Together with the mode bit, these ensure that a faulty display program cannot overwrite the alarm thresholds, the alarm task's stack, or the kernel.

9.5 Alternative concept and complexity analysis – polling instead of interrupts

If the monitors used polling (checking the sensor in a loop) instead of interrupts, what would the solution be?

Model (assumptions). A sensor produces a new reading every 2 ms (500 Hz). Checking the sensor and comparing the value with its limits costs **c = 2 µs** of CPU time (estimate). The alarm deadline is 10 ms from the moment the abnormal reading is available.

(a) Worst-case delay before an abnormal reading is noticed

Method	Worst-case delay to *notice*	Explanation
Polling every T	**T** (the reading may appear just after a poll) + c	Delay is determined by the programmer's polling interval and by whatever else the loop does

Method	Worst-case delay to *notice*	Explanation
Polling with T = 5 ms	≈ 5 ms; leaves 5 ms for everything else	Barely within the deadline; any other work in the loop breaks it
Polling with T = 1 ms	≈ 1 ms	Safe for this deadline but see cost below
Interrupt-driven	**Interrupt latency** + ISR start ≈ 5–50 µs (typical for an RTOS; bounded)	The hardware signals the CPU the moment the data is ready

With an interrupt-driven design, the end-to-end budget can be written down:

Step	Typical bound (estimate)
Interrupt latency	20 µs
ISR (read sample, post message)	10 µs
Context switch to alarm task	5 µs
Limit check and alarm decision	100 µs
Output to speaker or display driver	50 µs
Total	**≈ 185 µs, about 1.9% of the 10 ms deadline**

The interrupt design thus leaves a safety margin of more than 50 times; the polling design with T = 5 ms leaves a margin of two.

(b) CPU time and power wasted

Method	CPU load	Power
Busy-wait polling (loop checks continuously)	**100%** of the CPU, doing nothing useful; no time left for display or network	Processor always active, e.g. 0.5 W (estimate)
Timed polling at T = 1 ms	c / T = 2 µs / 1 ms = **0.2%**, and one timer wake-up per ms	Low, but the CPU wakes 1,000 times per second even when there is no event
Timed polling at T = 100 µs (to reduce delay)	c / T = **2%**	Wake-ups 10,000 times per second
Interrupts	Work only when data arrives: 500 samples/s × (10 + 5 + 100) µs ≈ **5.8%** of the CPU for a full sample-processing path; **idle otherwise**	The CPU sleeps (wait-for-interrupt state, e.g. 20 mW, estimate) between events

Evaluation: polling is **not** inherently slow, and a timer-driven periodic task *is* the standard way to sample a sensor at a fixed rate in an RTOS. The point is that the *alarm path* must not depend on a polling loop whose interval and other work the programmer must keep below the deadline. The 100% busy-wait loop is clearly unacceptable (no CPU left, high power, hot enclosure). Interrupts give the lowest, bounded delay and the lowest idle power, which also matters when a monitor runs on its backup battery.

Verdict for Task 8

Monitors use the **mode bit and privileged instructions** (so that no application can disable interrupts), **MMU or MPU** protection (so that faults stay inside one task), an **interrupt-driven sensor path** with bounded latency, a **timer** for periodic sampling and a **hardware watchdog** as a last safeguard.

10. Task 9 – Evaluate the Memory Hierarchy and I/O Systems

10.1 The memory hierarchy

Storage forms a pyramid: the faster a level is, the smaller and more expensive it is. Typical values (order of magnitude):

Level	Typical access time	Typical size	Volatile?	Managed by
Registers	≈ 0.3 ns	about 1 KB	Yes	Compiler
L1 cache	≈ 1 ns	32–64 KB per core	Yes	Hardware
L2 cache	≈ 4 ns	256 KB–2 MB per core	Yes	Hardware
L3 cache	≈ 15 ns	8–64 MB, shared	Yes	Hardware
Main memory (DRAM)	≈ 80–100 ns	8 GB–2 TB	Yes	OS (virtual memory)
NVMe SSD	≈ 20–100 µs	0.5–8 TB	No	OS, file system
Hard disk	≈ 5–10 ms	1–20 TB	No	OS, file system
Network storage (LAN)	≈ 0.5–5 ms	many TB	No	OS, storage system
Remote / cloud storage	tens of ms	practically unlimited	No	Cloud service

The gap between neighbouring levels is large: DRAM is about **1,000 times faster than an SSD** and a hard disk is about 100 times slower again. The hierarchy works because of **locality**: programs tend to reuse recently used data (temporal) and nearby data (spatial), so a small fast level holds most of what is needed.

10.2 Where each kind of data should be kept

Data	Where	Why
Monitor readings – the live stream	**RAM** ring buffer in the monitor (a few seconds to minutes)	The alarm needs nanosecond-to-microsecond access; no storage device may sit in the alarm path
Monitor trends (hours–days)	Local **flash/SSD** on the monitor, **and** streamed to the central database	Survives a network outage; limited writes to preserve flash
Active patient records (admitted patients)	**DRAM buffer cache** on the DB server, backed by **NVMe SSD** (replicated across cluster nodes)	Record look-ups at memory speed; durable on the SSD; the cluster protects against node loss
Old patient records	**Hard disks / network storage**, then archive (tape or cloud object storage)	Rarely read; cheaper per GB; slower access is acceptable
MRI image being processed	**DRAM** (the working set, often several GB), loaded by DMA	Processing speed depends on memory bandwidth
MRI raw and reconstructed studies (recent)	**NVMe SSD** array on the imaging server	High sequential bandwidth for large files
MRI archive	Network storage (a picture-archiving system) with a backup copy at a partner site	Large (hundreds of MB per study) and rarely read after a few weeks; durable

Data	Where	Why
Backups	Different building or site: disk, tape or cloud	Protection from fire, ransomware and site disasters

10.3 Programmed I/O, interrupt-driven I/O and DMA

Method	How it works	CPU involvement	Good for
Programmed I/O (polling)	The CPU repeatedly checks a device status register and moves each word itself	100% while waiting and copying	Tiny, fast, simple devices when little else is running
Interrupt-driven I/O	The CPU starts the operation and continues with other work; the device interrupts when a word or a block is ready; the ISR moves the data	An interrupt for each unit (word or block); the CPU copies the data	Low-volume, irregular devices: keyboard, sensor sample, serial port
Direct memory access (DMA)	The CPU programs a **DMA controller** with the source, destination address and byte count; the controller moves the whole block directly between the device and memory, stealing a few bus cycles; one interrupt when finished	Setup plus **one interrupt per block**	High-volume transfers: disk, network, **MRI scanner**

Choice for each device:

Device	I/O method	Reason
Monitor sensors	**Interrupt-driven** (timer or data-ready interrupt), optionally DMA for the ECG sample block	Low volume (about 500 samples/s) but needs immediate response
MRI scanner and storage	**DMA** (with scatter–gather descriptors)	Huge transfers; CPUs must be free for processing
Database disks and network	**DMA** (with interrupt coalescing)	Block transfers; thousands of requests per second
Tablet touch screen	**Interrupt-driven**	Events are rare; the CPU can sleep between touches, saving power
Workstation keyboard/mouse	**Interrupt-driven**	Irregular events

10.4 The effect of caching on the speed of record look-up

The **effective access time** with a cache is: EAT = h × t_cache + (1 − h) × t_miss, where *h* is the hit ratio. Using DRAM (0.1 µs = 100 ns) as the cache and an NVMe SSD (100 µs) as the backing store:

Hit ratio h	EAT with SSD behind the cache	Speed-up over SSD alone
0% (no cache)	100 µs	1×
90%	0.9 × 0.1 + 0.1 × 100 = **10.09 µs**	9.9×
99%	0.99 × 0.1 + 0.01 × 100 = **1.10 µs**	91×
99.9%	0.999 × 0.1 + 0.001 × 100 = **0.20 µs**	500×

If the backing store were a hard disk (8 ms = 8,000 µs), a 99% hit ratio would give 0.99 × 0.1 + 0.01 × 8,000 ≈ **80 µs**, about 100 times faster than uncached disk, but still 70 times slower

than the SSD with the same hit ratio. Two lessons for the hospital: (1) **even a 1% miss rate dominates the average**, so the design should keep the *working set* (records of currently admitted patients, which is a small fraction of all records) entirely in RAM; and (2) the SSD is worth its price in the miss path. Access to patient records is highly local (the same patient's chart is read many times during a shift), so a hit ratio above 99% is realistic for the active set.

10.5 Alternative concept and complexity analysis – interrupt-driven I/O (one interrupt per word) instead of DMA for MRI images

If the MRI scanner transferred images with interrupt-driven I/O (one interrupt per word) instead of DMA, what would the solution be?
Assumptions. Image size: 500 MiB = 524,288,000 bytes. Word size: 8 bytes (64-bit). Cost of one interrupt including the ISR that copies the word: **2 µs** (typical range 1–5 µs). DMA block (descriptor) size: 64 KiB.

(a) Number of interrupts and CPU time

Item	Interrupt-driven, one per word	DMA
Number of interrupts	524,288,000 / 8 = **65,536,000**	524,288,000 / 65,536 = **8,000**
CPU time spent on transfer	65,536,000 × 2 µs = **131 s** (about 2 min 11 s)	8,000 × 2 µs = **16 ms**, plus setup of each descriptor (a few µs each): well under **0.1 s**
Reduction	–	about **8,192 times fewer interrupts** (the ratio of 64 KiB to 8 bytes)
Maximum sustainable transfer rate per core	8 bytes / 2 µs = **4 MB/s**	Limited by the bus and device: 1 GB/s or more

The consequence is decisive: one CPU core doing nothing but servicing word interrupts could move only about **4 MB per second**, so a 500 MiB image would need over two minutes of full CPU time, while a modern scanner link delivers hundreds of MB/s. At 1 GB/s the interrupt rate would be 125 million per second, which is **250 times more than one core can service**. Meanwhile the CPU could not process any images. With DMA the same transfer takes the time given by the link (500 MiB at 1 GB/s ≈ 0.5 s) while the CPUs compute.

(b) Memory buffers needed

	Interrupt-driven	DMA
Device-side buffer	A small FIFO (for example 16 words = 128 bytes) must be emptied within microseconds, or data are lost	Device FIFO plus descriptors
Kernel buffer	Small (a few KB), as each word is copied into the destination as it arrives	A ring of **pinned (non-swappable) DMA buffers**, for example 2–4 buffers of 64 KiB–1 MiB for double or multiple buffering, or direct DMA into the final image buffer
Total memory for the image	500 MiB in any case if the image is to be processed in memory	500 MiB in any case

DMA needs a little more *kernel* memory (pinned buffers and descriptor tables, typically under a few MB) but saves enormous amounts of **CPU time**, which is the scarcer resource on the imaging server. The extra buffers cost far less than the 131 s of CPU time they avoid.

Verdict for Task 9

Use **DMA** for the MRI scanner, the database storage, and the network cards; keep **interrupt-driven** I/O for low-rate devices such as sensors and touch screens; keep the active records in **DRAM** over an **NVMe SSD**; and place old studies on cheaper, slower storage with off-site backup.

11. Task 10 – Measure and Compare

11.1 Experimental set-up

All programs are in Appendix A. The measurements were repeated to reduce noise: the buffer-size experiment and the process/thread experiment were each run **7 times**, and the **median** is reported (the minimum and maximum show the spread). The earlier single-run table in Section 7.6 is kept as a first look, and the comparison below shows why one run is not enough.

Item	Value
Platform	Linux 6.18, x86-64, 1 virtual CPU, sandboxed virtual machine
Data	16 MiB test file in the page cache (so storage-device speed is excluded and only the cost of system calls and copying is measured)
Timers	`clock_gettime(CLOCK_MONOTONIC)` for real time; `getrusage` for user and system CPU time (the same quantities the `time` command prints as `user` and `sys`)
`strace`	Not installed. Call counts are therefore taken from the program's counters and from the exact relationship n/b, which the counter results confirmed

11.2 Number of system calls for different buffer sizes

Buffer b	read() calls (counted)	write() calls	Total I/O system calls (≈)	Source
1 byte (10,000-byte file)	10,001	10,000	20,001	program output (Task 6)
512 bytes (10,000-byte file)	21	20	41	program output
4,096 bytes (10,000-byte file)	4	3	7	program output
16 bytes (1 MiB file)	1,048,576	1,048,576	≈ 2.1 million	benchmark counter
64 bytes (16 MiB)	262,144	262,144	≈ 0.52 million	benchmark counter
512 bytes (16 MiB)	32,768	32,768	≈ 65,500	benchmark counter
4,096 bytes (16 MiB)	4,096	4,096	≈ 8,200	benchmark counter
65,536 bytes (16 MiB)	256	256	≈ 512	benchmark counter
1 MiB (16 MiB)	16	16	≈ 32	benchmark counter

Check against theory: every count equals ⌈n/b⌉ (plus 1 for the end-of-file read in the 10,000-byte tests). The prediction and the measurement agree exactly, because each loop iteration is one `read()` and one `write()` system call.

11.3 Time in user mode and kernel mode

First run (Section 7.6), CPU time by mode:

Buffer b	Real (s)	User (s)	System (s)	Share of CPU time in kernel mode
1 byte (1 MiB file)	0.4928	0.1194	0.3718	76%
16 bytes	0.5276	0.1184	0.4049	77%
64 bytes	0.1364	0.0282	0.1005	78%
512 bytes	0.0423	0.0009	0.0231	96%
4,096 bytes	0.0193	0.0005	0.0097	95%
65,536 bytes	0.0299	0.0003	0.0090	97%
1 MiB	0.0126	0.0004	0.0053	93%

Repeated runs (median of 7, real time for a 16 MiB copy):

Buffer b	Median real (s)	Min (s)	Max (s)	Throughput at median	System calls
512 bytes	0.0376	0.0300	0.0583	≈ 450 MB/s	≈ 65,500
4,096 bytes	0.0243	0.0184	0.0307	≈ 690 MB/s	≈ 8,200
16,384 bytes	0.0111	0.0096	0.0240	≈ 1.5 GB/s	≈ 2,050
65,536 bytes	0.0075	0.0067	0.0111	≈ 2.2 GB/s	≈ 512
262,144 bytes	0.0071	0.0066	0.0092	≈ 2.4 GB/s	≈ 128
1,048,576 bytes	0.0075	0.0070	0.0082	≈ 2.2 GB/s	≈ 32

Interpretation.

- With tiny buffers **three quarters of all CPU time is spent in kernel mode**: the program is not computing, it is crossing the user–kernel boundary. The user time with 1-byte buffers (0.12 s for 1 MiB) is itself mostly the library-call and loop overhead of two calls per byte.
- With buffers of 512 bytes and above, user time almost vanishes (under 1 ms) and the remainder is kernel copying.
- The repeated runs show a clean curve: time halves each time the buffer grows 4× up to 64 KiB (0.0376 → 0.0243 → 0.0111 → 0.0075 s), then **the curve is flat from 64 KiB to 1 MiB** (0.0071–0.0075 s, differences within the spread). The remaining time is data copying, which no buffer size can remove.
- The single-run table suggested that 64 KiB (0.0299 s) was slower than 4 KiB (0.0193 s). The medians **contradict** that: 64 KiB is about 3.2 times faster than 4 KiB. This is a direct lesson in measurement: a single run on a shared virtual machine can mislead, so a conclusion must rest on repeated runs.

11.4 Time to create 1,000 processes and 1,000 threads

Measurement (1,000 repetitions unless stated)	Result (median or single run as stated)	Per item
fork() + _exit + waitpid, one after the other	0.0527 s (median of 7)	52.7 µs
vfork() + _exit + waitpid	0.0086 s (single run)	8.6 µs
Thread create + join, one after the other	0.0077 s (median of 7)	7.7 µs
fork() + exec("/bin/true") + waitpid	0.7024 s (single run)	702 µs
1,000 processes alive at the same time (fork, each sleeps 1 s)	**0.1098 s** (single run)	110 µs
1,000 threads alive at the same time (each sleeps 1 s)	**0.0144 s** (single run)	14.4 µs

The ratio for equal conditions is 0.1098 / 0.0144 ≈ **7.6**: creating a process is **about 7 to 8 times slower** than creating a thread; for sequential create-and-join, the ratio is 52.7 / 7.7 ≈ **6.8**.

Effect of the size of the parent on `fork()` (200 forks each, parent has touched the stated memory):

Parent memory touched	Time per fork + exit + wait
1 MiB	83 µs
8 MiB	156 µs
64 MiB	764 µs
512 MiB	4,960 µs (≈ 5 ms)

From 1 MiB to 512 MiB the cost grows about 60 times; the slope is about **9.5 µs per additional MiB** of touched memory (the cost of copying page-table entries and marking pages copy-on-write). Extrapolating linearly, a parent with 10 GB resident would need about **100 ms** per `fork`. Thread creation does not depend on the size of the program because nothing is copied.

11.5 Why threads are cheaper: what is shared

Resource	Process created by fork	Thread created by `pthread_create`
Address space (code, heap, globals)	**Copied** (page tables duplicated; pages shared copy-on-write)	**Shared** – nothing to copy
Open files, signal handlers	Copied (descriptor table duplicated)	Shared
Registers and program counter	Own	Own
Stack	Own (copy of parent's)	Own, newly allocated (default 8 MiB *virtual*; ulimit -s was 8192 KB)
Kernel structures	New task structure, new memory descriptor, new page tables, new kernel stack	New task structure and kernel stack only
Typical memory	Page tables ≈ 0.2% of the parent's resident size plus kernel stack (≈ 16 KiB) plus pages that become private after writes	Kernel stack (≈ 16 KiB) plus the user stack pages actually touched (a few KiB)

The time difference is explained by what has to be built: the thread needs a scheduler entry and a stack; the process also needs its own copy of the memory map and page tables. (Per-process and per-thread memory figures in this table are typical values; memory use was **not** measured.)

11.6 Do the measurements support or contradict the earlier judgements?

Earlier judgement	Evidence	Verdict
A system call is about 50–100 times dearer than a function call (Task 6)	106 ns per null system call; 0.34 ns per trivial loop step; a normal function call (not separately measured) is typically 1–2 ns	**Supported** (the function-call figure is a literature value)
Call count falls as n/b (Tasks 5, 6)	Counters equal ⌈n/b⌉ in every row	**Supported exactly**
Large buffers are better, with diminishing returns (Task 6)	Time halves per 4× buffer up to 64 KiB, flat after	**Supported**

Earlier judgement	Evidence	Verdict
"64 KiB is a sensible backup buffer"	Medians: 64 KiB = 0.0075 s; 256 KiB = 0.0071 s; 1 MiB = 0.0075 s	**Supported** by medians, but the first single run seemed to **contradict** it; repeated runs resolved it
mmap removes system calls but is not automatically faster (Task 6)	mmap copy 0.0378 s (single run) versus 0.0075 s for read/write with 64 KiB	**Supported**: mmap was about 5 times slower here
Library buffering cuts calls by about 4,096 for byte-wise access (Task 5)	Derived from the buffer-size formula; stdio itself was not benchmarked	**Consistent but not directly measured**
fork + exec costs much more than fork alone (Task 7)	702 µs versus 53 µs, factor 13	**Supported**
Threads are much cheaper than processes (Tasks 7, 10)	7 to 8 times cheaper in creation	**Supported**
"fork of a 10 GB parent costs milliseconds" (Task 7, stated loosely)	Measured slope extrapolates to **about 100 ms**	**Partly contradicted.** Copy-on-write is much cheaper than copying the memory, but 100 ms is not negligible; for a large database process the shell should use vfork/posix_spawn, which strengthens that recommendation
Microkernel messages cost 1–5 µs (Task 3)	Not measured (no microkernel available)	**Untested** – a literature value
Interrupt latency of tens of µs on an RTOS (Task 8)	Not measured (no real-time hardware)	**Untested** – the design budget in Task 8 depends on it and must be verified on the real monitor hardware

11.7 Alternative concept and complexity analysis – threads instead of processes in the measurements

If threads were created instead of processes for the measurements, what would the result be?

	1,000 processes	1,000 threads	Ratio
Creation time, sequential (median)	52.7 ms	7.7 ms	6.8×
Creation time, all alive together	109.8 ms	14.4 ms	7.6×
Creation time if the parent were 512 MiB	about 5 s (1,000 × 4.96 ms)	still about 8–14 ms	about 350–600×
Memory per item (typical)	Kernel stack ≈ 16 KiB + page tables (≈ 2 MiB per GiB of parent) + pages made private by writes	Kernel stack ≈ 16 KiB + touched stack pages; 8 MiB of *virtual* address space reserved but not committed	Threads use a fraction
Isolation	Complete	None	–

Explanation using what each one shares. Threads share the address space, the files and the signal table, so the kernel only needs a new execution context. A process shares nothing by default; fork must duplicate the memory map and page tables (a cost that grows with the parent's size) and the file-descriptor table. Therefore threads are cheaper and faster to create, to switch between (the same page tables stay in use, so the TLB need not be flushed) and to communicate between (memory is directly shared). The price is lost isolation: one thread's bug can corrupt or terminate all the others.
Consequence for the hospital design. Use **threads** within one trusted program where speed and sharing are needed (MRI processing workers, the database's connection handlers, which are a single trusted program), and **processes** where isolation is a safety or security requirement (separate services on the monitor, separate commands run by the shell, separate users' sessions).

12. Task 11 – Recommend and Defend the Operating-System Plan

12.1 The recommended plan

Environment	OS type	Kernel structure	Services kept	Protection	I/O method	Example product class
E1 Bedside monitor	Hard real-time OS	Microkernel (drivers and stacks in user space)	Fixed alarm UI, sensor I/O, one authenticated channel, watchdog; no shell, no general file system, no virtual memory	Mode bit, MMU/MPU, signed firmware, locked ports	Interrupt-driven sensors (bounded ISR); DMA for sample blocks	QNX-class or certified RTOS
E2 Nurse tablet	Handheld OS	Layered stack on a modular (Linux-based) or hybrid kernel	Touch UI, sandboxed apps, Wi-Fi/VPN, power management	App sandboxes, full-disk encryption, device management with remote wipe, biometric/PIN lock	Interrupt-driven; DMA for storage and radio	Android-class managed devices
E3 Doctor workstation	Desktop (time-sharing) OS	Hybrid (Windows) or modular monolithic (Linux)	Full GUI, networking, printing	Accounts, central management, antivirus, least privilege, automatic updates	Interrupt-driven and DMA (standard)	Windows or Linux desktop, centrally managed

Environment	OS type	Kernel structure	Services kept	Protection	I/O method	Example product class
E4 Database server	Clustered server OS (2–3 nodes with fail-over)	Modular monolithic kernel	Journaling file system, caching, accounting/ audit, resource limits	ACLs, mandatory access control, database roles and row-level security, encryption at rest/in transit	DMA with caching; interrupt coalescing	Linux cluster with replicated NVMe storage
E5 Imaging server	Symmetric-multiprocessor server OS	Modular monolithic kernel	Large-file file system, threads, NUMA-aware scheduling	Same hardening as E4; image-access policy	**DMA**, scatter–gather	Linux on multi-core server with NVMe array
E6 Partner link	Distributed system via gateway servers	Modular monolithic gateway OS	Communication, audit, queueing	Mutual TLS authentication, encryption, message signing, audit log	Interrupt/DMA network	Hardened Linux gateways with an interface engine for standard record formats

What the single-OS idea gets right and where it is kept. The servers (E4, E5, E6) and, optionally, the workstations can share **one family** (Linux) and one set of administration tools, so the IT team keeps most of its saving. Only the **monitors** (hard real-time) and **tablets** (handheld) differ, because their requirements cannot be met otherwise.

12.2 Evidence from the analysis and measurements

Decision	Evidence
Real-time microkernel for monitors	Task 1: unbounded versus bounded alarm delay. Task 3: microkernel IPC costs ≈ 15 µs on a five-request path (0.15% of the 10 ms budget) but isolates faulty drivers. Task 8: interrupt-driven path totals ≈ 185 µs (1.9% of deadline). Measured null system call 106 ns shows service calls are small compared with the deadline; the larger risk is non-determinism, not average speed.
Handheld OS for tablets	Task 4: 12 h versus 6 h battery in the worked estimate; 15–25 GB less storage.
Cluster for database	Task 2: availability ≈ 99.9998% versus ≈ 99.94% for a single machine (≈ 1 min versus ≈ 5.5 h per year, estimated).
SMP + DMA for imaging	Task 2: Amdahl speed-up (15× at 64 cores for p = 0.95). Task 9: interrupt-per-word would take 131 s of CPU for 500 MiB; DMA needs 16 ms.

Decision	Evidence
64 KiB buffers for the backup program	Task 6 and Task 10: the 64 KiB median is 3.2× faster than 4 KiB and equal to 1 MiB; 512 system calls instead of 8,200.
fork+exec (or posix_spawn) for command execution, threads for workers	Task 10: process creation 7–8× the cost of a thread, but isolation is essential between commands; fork cost grows about 9.5 µs per MiB of parent.
Protection layers	Task 4 and Task 8: hardware memory protection, mode bit, permissions, row-level controls and audit.

12.3 Limitations and risks of the plan

Risk or limitation	Consequence	Mitigation
Four OS families to administer	More training and more patch streams	Standardise servers on one Linux family; use management tools for each class; keep the monitor OS image small and signed
Monitor updates	Updating a device in use is a safety event	Update only when the monitor is idle or off-bed, use dual-bank firmware with roll-back, and stage by ward
Real-time guarantees depend on hardware	Cache effects, power-saving states and shared buses can break analysis	Verify worst-case latency on the actual hardware; keep power states that add wake-up delay off in the alarm path
Cluster split-brain, shared-storage failure	Data corruption or total outage	Quorum with a witness node, fencing, replicated storage with RAID
Monolithic kernel on the servers	A bad driver takes down a node	Cluster fail-over; use only certified drivers; staged kernel updates
Lost or stolen tablets	Privacy breach	Encryption, remote wipe, short timeouts
Single site for the cluster	A fire or flood takes out both nodes	Put nodes in separate rooms and replicate to a second site
Measurement limits (this report)	Test machine is a virtual one-CPU system; no strace; microkernel and interrupt latencies not measured	State clearly; repeat on target hardware before purchase; do acceptance tests on a prototype monitor
Staff skills	Unfamiliar RTOS	Vendor support and training; run a pilot ward first

12.4 Is the plan still suitable for a group of three hospitals?

Scale-up under the assumption that each partner hospital is about the same size as the first: roughly **1,500 monitors, 360 tablets and 240 workstations**.

Component	One hospital	Group of three	Change needed
Monitors	500	≈ 1,500	**None to the OS design.** The real-time OS is local to each device; what grows is the **fleet management** and the load on the central station

Component	One hospital	Group of three	Change needed
Tablets	120	≈ 360	Mobile-device-management server capacity; unchanged OS
Workstations	80	≈ 240	Central management and identity services must be federated across sites
Database	One 2-node cluster	Either one larger cluster at a primary site with a replica site, or one cluster per hospital with replication between them	**Modify**: add geographic replication, a consistent patient identifier across sites, and a plan for operation during link failure (each hospital must be able to work offline)
Imaging	One SMP server	One per site or a shared pool	Modest: images are large, so keep processing local and transfer only results
Partner link	Gateway to two partners	Becomes a **mesh** among three	The distributed design scales if a standard message format and central identity (federation) are used

Conclusion: the plan **remains suitable**. The device-level choices need no change; the **distributed** parts (database replication, identity, partner link) need extension, which Task 12 builds on.

12.5 Alternative concept and complexity analysis – moving everything into virtual machines on two large servers

If the whole system were moved to virtual machines on two large servers, instead of separate physical machines for each environment, what would the solution be?

A **hypervisor** runs on each physical server and divides it into virtual machines (VMs), each with its own guest OS. Workstations could be provided as virtual desktops; servers as VMs; the monitors and tablets, being physical devices at the bedside, cannot be moved into a server — only their *back-end services* can.

(a) Effect on response time for the monitors

- The monitors themselves must stay **physical devices**: a deadline of 10 ms cannot depend on a network round trip to a server room and a hypervisor.
- If the *processing* were moved to a VM, the delay would include the **network** (about 0.1–1 ms on a good LAN, with jitter) and the **hypervisor scheduling** of the virtual CPU. A virtual CPU can be descheduled for a hypervisor time slice (often milliseconds), and virtual timers add jitter; real-time-tuned hypervisors reduce this to tens or hundreds of microseconds with pinned CPUs, but the bound is looser and harder to prove than on bare metal. Virtualised monitor logic would therefore **put the 10 ms guarantee at risk**.
- For the **other** services (databases, imaging, workstations), the response time is soft and tolerates the small extra delay.

(b) Memory and processor overhead of virtualisation

Overhead	Typical value (estimate)	Effect on this hospital
CPU overhead of a hardware-assisted hypervisor	2–5% for compute work; 5–15% for I/O-heavy work without paravirtual drivers	Acceptable for most services; the database's heavy I/O needs paravirtual or pass-through devices
Memory per VM	A full guest OS (0.5–1 GB) in each VM, partly reduced by page sharing	10 VMs cost about 5–10 GB just for guest operating systems
Extra memory-translation cost	Two levels of page tables (guest and host) make a TLB miss more expensive	Slower for memory-intensive MRI processing
Device I/O	DMA through the hypervisor (or direct pass-through)	MRI DMA should use **pass-through**; otherwise it adds copies and CPU load

(c) Effect on reliability

Aspect	Separate machines	Everything on two servers
Failure domain	One failure affects one function	One physical server failure affects **every VM on it**: database node, imaging, gateway and workstations together
Recovery	Cluster fail-over in seconds for the database	HA restart of VMs on the other host in **minutes** (a VM must boot), unless the VM pair is itself a cluster on *different* hosts
Both servers lost	Unlikely	Total outage (common power, cooling, or hypervisor bug)
Isolation	Physical	Logical: a hypervisor flaw can break isolation between environments
Patching	Each machine separately	One hypervisor patch affects all VMs: needs live migration to avoid downtime

Using the same approximate figures as in Task 2: if two hosts each have 99.96% availability and VMs restart in about 2 minutes after a host failure, a host failure rate of about 0.9 per year per host gives 1.75 failures × 2 min ≈ **3.5 minutes per year** for the services — acceptable for workstations and management services, but **about three times worse** than the ≈ 1 minute of the physical cluster, and a double host failure would stop everything.

13. Task 12 – Day-4 Design Challenge: Re-evaluate and Adapt the Design

Changed requirement. The hospital adds (1) remote monitoring devices in patients' homes that send readings over the internet, and (2) a cloud service that holds patient records shared with the partner hospitals. The decision to be made for each part of the original design is one of **RETAIN**, **MODIFY** or **REDESIGN**.

13.1 Part A – Impact analysis

A.1 What is new

New element	Key properties	Why it matters to an OS designer
E7 Home monitoring device	In an uncontrolled place; consumer broadband or mobile data; battery or mains with no backup; physically accessible to anyone; maintained by the patient, not by IT	Needs an embedded real-time OS that is safe *without* a network, secure against physical tampering, power-frugal and remotely updatable
E8 Edge / ingestion tier	Receives readings from many homes over the internet, authenticates devices, queues messages	A distributed server system; must handle bursts and malicious traffic
E9 Cloud records service	Run by a third party; shared with partner hospitals; replicated across data centres	Moves trust outside the hospital: the hospital no longer controls the OS, the hardware or the administrators
Internet path	Round-trip time of tens to hundreds of ms, with jitter, loss and outages	Makes a hard 10 ms deadline across the network **impossible**

A.2 Which operating-system types are affected

Original component	Affected?	How
E1 Bedside monitor (hard real-time microkernel)	**Not affected** locally; becomes the **model** for E7	The OS type is right for a home device too; what changes is the environment around it
E2 Nurse tablet (handheld OS)	Slightly	Must now reach the cloud records securely from outside the hospital Wi-Fi; may also be used on home visits
E3 Doctor workstation (desktop OS)	Slightly	Records come from the cloud as well as the local database; needs single sign-on and multi-factor authentication
E4 Database cluster (clustered OS)	**Affected strongly**	The authoritative copy of the record is now shared; the cluster must replicate to a cloud service, and must keep working when the link fails
E5 Imaging server (SMP)	Little	Images stay local (too big to move); only reports or references go to the cloud
E6 Partner link (distributed gateway)	**Replaced**	Two point-to-point links become a multi-party federation through the cloud service

A.3 Which services are affected

Service	Impact
Communication	Now crosses an untrusted network at both ends; needs authenticated, encrypted channels, retries, store-and-forward, and time synchronisation
Protection and security	The perimeter model (a trusted hospital network with guarded links) no longer holds; every device and user must be authenticated every time (zero trust)
Accounting / audit	Must record accesses by partner staff and by cloud administrators, in a form the hospital controls

Service	Impact
Resource allocation	Cloud services share hardware among tenants (hospitals): quotas and isolation between tenants
Error detection	Must also detect lost messages, duplicated messages, out-of-order data and clock drift
Program execution / updates	Home devices need **over-the-air (OTA) updates** that can be rolled back; cloud services update continuously
File systems / storage	Buffers on the home device for outages; replicated storage in the cloud; encryption keys
User interface	Patients are non-experts: simple, with local alarm sounds and clear instructions

A.4 Which protection mechanisms are affected

Mechanism	Change
Mode bit, MMU/MPU, privileged instructions (Task 8)	**Unchanged** and still the base layer on every device and server
Passwords and local accounts (Task 4)	Insufficient: replaced by **device certificates** (stored in a secure element), **federated identity** and **multi-factor authentication** for people
File permissions / ACLs	Extended by **per-patient consent** and attribute-based access that follows the record into the cloud
Encryption	Moves from "nice to have" to **mandatory** in transit and at rest; hospital-held keys
Physical security	The home device can be stolen or opened: **secure boot**, signed firmware, encrypted storage, tamper detection
Audit	Tamper-evident logs held outside the cloud provider's control

13.2 Part B – Evaluation of the original design

The question: can the original plan support home devices and cloud records, in terms of **response time, reliability and security**?

B.1 Response time

Question	Analysis
Can the central system raise a 10 ms alarm for a home patient?	**No.** The internet round-trip time to a hospital or cloud region is typically 20–100 ms or more, with jitter, so the network alone breaks 10 ms. (Typical values, not measured here.)
Does the original monitor design help?	**Yes, if it is placed in the home.** A hard real-time device raises the alarm *locally* (a sound and a light) within the same few hundred microseconds as in the hospital; the network is used for a *notification* with a soft target of a few seconds.
Is a general-purpose OS on the home device acceptable?	No: the same argument as in Task 1 applies; unbounded scheduling delay on a device that is the patient's only protection.

B.2 Reliability

Availability of a chain of components in series is the **product** of their availabilities. Using illustrative (estimated) figures: cloud service 99.95% (about 4.4 h/year of downtime) and a single home or hospital internet link 99.9% (about 8.8 h/year):

Path	Calculation	Downtime per year
Original on-site cluster (Task 2)	–	≈ 1 minute
Hospital reaches the **cloud-only** record over one link	0.9995 × 0.999 = 0.99850	**≈ 13 hours**
Same with two independent links (99.9999% combined)	0.9995 × 0.999999 ≈ 0.99950	≈ 4.4 hours (limited by the cloud service)
Hospital using a **local cluster that keeps working** and synchronises with the cloud later	Local availability as before	≈ 1 minute for local care; the cloud may lag

The lesson is clear: a design that makes the hospital **depend on the cloud for care at the bedside** would cut availability by a factor of 100 to 800 compared with the on-site cluster, and the requirement "the database must never stop" would be broken. The original cluster must therefore **remain the authoritative source for in-hospital care**, with the cloud copy as a replica for sharing.

For home devices the reliability requirement adds **outage tolerance**. A device sampling one ECG lead at 250 Hz with 2 bytes per sample produces about 500 bytes/s, i.e. 43 MB per day. A 7-day local buffer needs only about 300 MB of flash, so a device can store everything through a multi-day outage and upload later.

B.3 Security

Aspect	Original design	Under the new requirement	Result
Trust boundary	Hospital network, two guarded partner links	Hundreds of homes, internet, cloud provider, partner staff	**Fails**: the boundary no longer exists
Device identity	Devices are in locked hospital areas on a managed network	Devices in homes can be stolen, copied or cloned	**Fails**: needs strong device identity and secure boot
Authentication of people	Local accounts and hospital login	Partner and remote users from anywhere	**Insufficient**: needs federation and multi-factor authentication
Updates	IT patches devices on site	Patient-owned devices must be patched remotely	**Fails**: needs signed over-the-air updates with roll-back
Data protection	Within one organisation	Shared with partners and held by a cloud provider; data-protection law (for example India's Digital Personal Data Protection Act, 2023, or the rules of the partners' countries) applies to the processing and transfer	**Needs** consent management, encryption with hospital-held keys and audit trail

Aspect	Original design	Under the new requirement	Result
Attack volume	Few external connections	Internet-facing ingestion endpoints are constantly scanned and attacked	**Needs** rate limiting, intrusion detection and hardened services

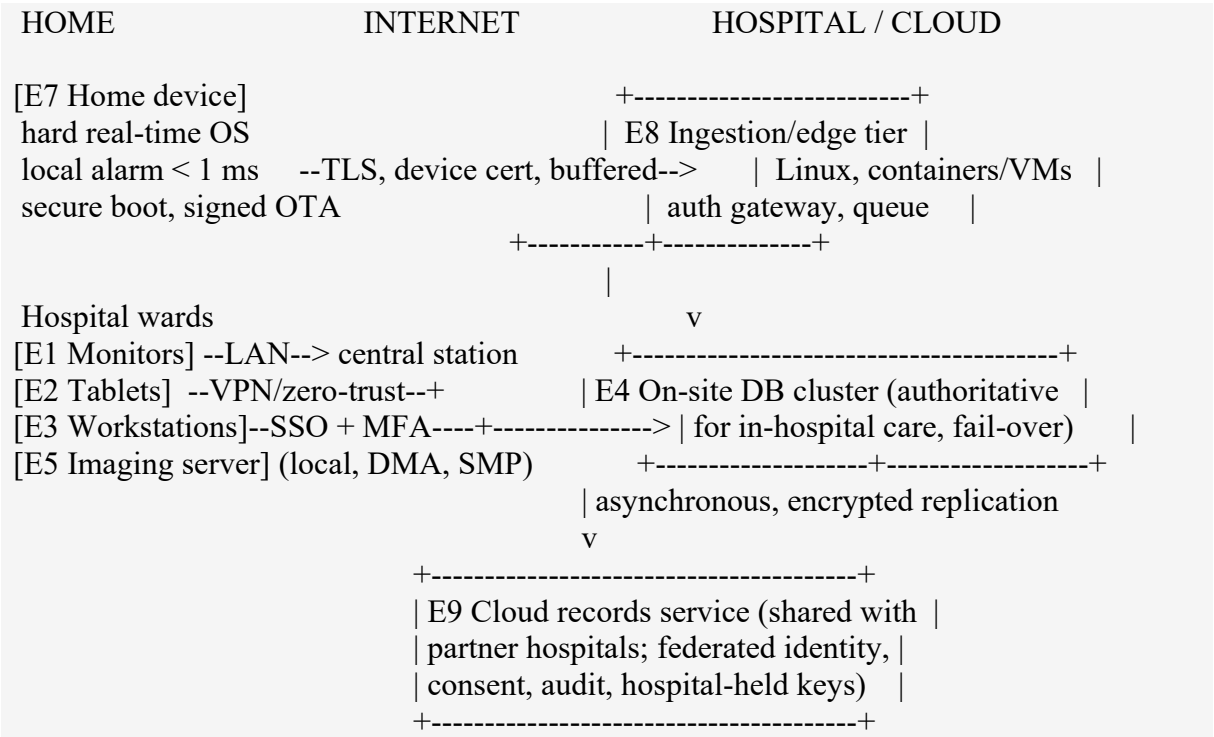
B.4 Verdict on the original design

Part	Decision	Reason
E1 monitor OS type and structure	**RETAIN** (and re-use for home devices)	Meets the deadline locally and isolates faults
E2 tablets, E3 workstations	**RETAIN** with **modification** of access (VPN or zero-trust gateway, MFA)	OS choice is still right; access paths change
E5 imaging server	**RETAIN**	Images are large and stay local
E4 database	**MODIFY**	Keep the cluster as authoritative for care; add replication to the cloud; add offline operation
E6 partner link	**REDESIGN**	Replace point-to-point links by federated, standards-based access through the shared cloud service
New E7, E8, E9	**ADD**	Not in the original plan

Overall: **the original design is modified, not discarded**. Only the partner link is redesigned; the new elements are added around it.

13.3 Part C – Adaptation or redesign

C.1 The revised architecture



C.2 Operating-system choices for the new environments

Environment	OS type	Structure	Services kept	Protection	I/O
E7 Home monitoring device	**Embedded hard real-time OS** (FreeRTOS with memory-protection unit, Zephyr, or a QNX-class microkernel if the hardware has an MMU)	Microkernel or small kernel with MPU-separated tasks	Sensor I/O, alarm output, buffered local store (a small flash log), authenticated TLS client, watchdog, OTA agent; **no** shell, **no** inbound ports, **no** general file system	Secure boot, signed firmware with A/B (dual-bank) update and roll-back, device certificate in a secure element, encrypted storage, tamper detection	Interrupt-driven sampling; DMA for radio and storage
E8 Ingestion/edge tier	**Distributed** server system	Modular monolithic kernel (Linux) with containers (namespaces, control groups) or lightweight VMs for tenant isolation	Communication, queueing, authentication, rate limiting, accounting	Mutual TLS, system-call filtering for each service, network policies, intrusion detection	DMA network, interrupt coalescing
E9 Cloud records service	**Distributed system** operated by a provider, with the hospital's own **tenant** and keys	Provider's hypervisor and kernels (outside our control); we choose isolation level (dedicated VMs for the database tier)	Replicated storage, consistent record identity, consent, audit	Customer-managed encryption keys, federated identity, per-patient consent, immutable audit log	Provider's
E4 Database (modified)	Clustered (unchanged) plus replication agent	Modular monolithic kernel	As before, plus replication and conflict handling	As before	As before
E6 Partner link (redesigned)	Federation through E9	–	Standards-based API with single sign-on between hospitals	Mutual TLS, signed tokens	–

C.3 How the earlier lessons are re-used

- **System calls (Task 5).** In the server tier each service is limited to the system calls it needs (a *system-call filter*, for example seccomp on Linux). A filter adds a small cost to each call; compared with the measured 106 ns per null system call, an extra tens of nanoseconds is acceptable for servers, and it shrinks the damage a compromised service can do. Home devices go further: they simply do not contain the code for services they do not need (Task 4).
- **Cost of system calls (Task 6).** The ingestion tier batches readings, as the backup program batches bytes: one message of 100 readings instead of 100 messages cuts per-message overhead by a factor of about 100, just as a 64 KiB buffer reduced the calls from millions to hundreds.
- **Processes and threads (Tasks 7, 10).** The ingestion workers are threads inside one trusted service (cheap, fast); different tenants and different services run in separate **processes or containers** (isolation). The 7.6-fold cost of a process over a thread is paid only where the isolation is needed.
- **Protection hardware (Task 8).** Every layer above still rests on the mode bit and memory protection; containers share a kernel, so for the most sensitive tenant data the design uses **VMs** (hypervisor-enforced separation) rather than containers.
- **Interrupts versus polling (Task 8).** The home device uses an interrupt-driven alarm path; the cloud side uses polling-style periodic *health checks* only where the delay is in seconds.
- **Memory hierarchy (Task 9).** Home buffers go to flash only in batches (to preserve flash life) while the live alarm data stay in RAM; cloud query caches keep the working set of active patients in memory.

C.4 Sizing the new load (estimate)

Assume 1,000 remote patients, each device sending a 1 KB summary every 10 seconds, with raw waveforms stored locally and uploaded only around alarm events.

Quantity	Calculation	Result
Message rate	1,000 / 10 s	**100 messages/s**
Data rate	100 × 1 KB	about 100 KB/s
Data per day	100 KB/s × 86,400 s	about **8.6 GB/day**
Per-message work on one core	100 µs (estimate)	100 × 100 µs = 10 ms per second = **1% of one core**

The volume is small for the servers: the difficulty is **security, reliability and operations**, not raw throughput. A burst after an internet outage (all devices uploading buffered data at once) can multiply the rate by 10–100, so the ingestion tier needs a queue and rate limiting.

C.5 Synchronising the on-site cluster with the cloud

1. In-hospital care always uses the on-site cluster (authoritative, fail-over in seconds).
2. Each committed change is written to a replication log and sent asynchronously, encrypted, to the cloud service. If the link is down, the log grows on local disk and catches up when the link returns.
3. Partner hospitals read from the cloud copy; when they need to write, the change goes through the cloud to the owning hospital, which applies it (or a defined conflict rule), so that each patient record has one **owner** at a time.
4. Delay between a change and its visibility to partners is a **soft** requirement (seconds to minutes); the on-site care path never waits for the cloud.

13.4 Part D – Evidence-based defence

D.1 Options compared

Option	Description
A. Retain unchanged	Treat home devices like extra ward monitors over a VPN; move all records to the cloud and use the cloud as the only database
B. Modify (recommended)	Keep the original device-level design; add an RTOS home device with local alarm; add an ingestion tier; keep the on-site cluster as authoritative and replicate to the cloud; replace the partner link by a federation
C. Redesign everything	Cloud-native: process the alarms in the cloud, run all servers as cloud VMs, retire the on-site cluster

Criteria and weights (weight 5 = most important): patient safety 5, availability 4, security 5, cost 2, administration simplicity 2. Scores 1–5 (5 = best) are **judgements** from the analysis above, not measurements.

Criterion (weight)	A. Retain	B. Modify	C. Redesign
Patient safety (5)	3	5	2
Availability (4)	2	4	3
Security (5)	1	4	4
Cost (2)	4	3	2
Administration simplicity (2)	4	3	2
Weighted total (out of 90)	15 + 8 + 5 + 8 + 8 = **44** (49%)	25 + 16 + 20 + 6 + 6 = **73** (81%)	10 + 12 + 20 + 4 + 4 = **50** (56%)

Sensitivity check. If the weight of cost is doubled from 2 to 4 (maximum 100), the totals become A = 52, B = 79 and C = 54: option B remains first. The result changes only if cost is weighted more heavily than patient safety, which is not defensible for a hospital.

D.2 Defence of the decision, with the evidence

1. **A 10 ms alarm cannot cross the internet.** Typical round trips of 20–100 ms exceed the deadline many times over; the alarm must be raised on the home device by a real-time OS, exactly as in Task 1 and Task 8 (interrupt path of about 185 µs).
2. **A cloud-only record breaks "never stop."** Serial availability gives about 13 hours of downtime per year against about 1 minute for the on-site cluster (Task 2 and Part B above). The cluster is therefore retained and the cloud is a replica.
3. **Isolation is worth its price.** Task 10 measured process creation at about 7–8 times a thread; this cost is acceptable for tenant and service separation, while threads remain for workers within a service.
4. **Home devices are small and must remain simple.** Task 4 and Task 3 showed that removing services and using a microkernel or small RTOS shrinks the code and the attack surface; for a device that is physically in an untrusted place this is the most important security measure after secure boot.
5. **Measurement honesty.** The figures for internet latency, availability of cloud services and of links, microkernel IPC and interrupt latency are typical or estimated values, **not** measured in this study; the measured values are the system-call counts, buffer-size timings, system-call cost, and process/thread creation costs. The design decision does not depend on small differences in those numbers: even if the real figures were ten times better, a networked alarm would still have no guaranteed deadline, and a serial cloud dependency would still lower availability.

D.3 Trade-offs accepted

Trade-off	What is given up	Why it is accepted
More components (E7, E8, E9)	Simplicity	Needed to meet safety and security
Two copies of the record	Extra storage and the risk of conflicts	Gives bedside availability and sharing; solved by one owner per record
Cloud provider dependency	Some control; possible lock-in	Standard interfaces (FHIR, TLS, OIDC), hospital-held keys, an exit plan and an up-to-date export
Embedded RTOS fleet in homes	Remote-support burden	Signed OTA with roll-back and device health reporting
Higher cost than option A	Budget	Option A fails on safety and security

D.4 Risks that remain, and what would change the decision

Residual risk	Mitigation
Compromised home device used to attack the hospital	Treat devices as untrusted; per-device authorisation; anomaly detection; rate limiting
Failure of the OTA update (bricked devices)	A/B partitions, staged roll-out of 1%, 10%, 100%
Cloud provider outage	On-site cluster continues; test an exercise every quarter
Replication conflicts between hospitals	One owning hospital per record; audited conflict rules
Data-protection compliance	Consent records, data-residency control, hospital-held keys, regular audit

Would change the decision: (a) a certified real-time cloud-edge product whose guarantees are independently proven (this could allow a lighter local device); (b) a regulator that requires the authoritative record to be in a named cloud region (the roles of cluster and cloud would be reversed, with a larger local cache); (c) measured latency from home to hospital consistently below 5 ms (very unlikely over the public internet), which would still not give a guarantee.

13.5 Final decision (one paragraph)

Retain the device-level operating-system choices (hard real-time microkernel for monitors, handheld OS for tablets, desktop OS for workstations, SMP server for imaging) and the on-site database cluster; **modify** the database (replication to the cloud and offline operation) and user access (VPN or zero trust, single sign-on, multi-factor authentication); **redesign** the partner link as a federation through the cloud records service; and **add** an embedded real-time OS on the home devices, a distributed ingestion tier and a cloud tenant with hospital-held keys. This keeps the guarantees that matter for patient safety (local deadlines and local availability) while adding the sharing and reach that the hospital wants.

14. Appendices

Appendix A – Source code used for the measurements

A.1 Buffer-size benchmark (core of the program used in Tasks 6 and 10)

```
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
```

```
#include <time.h>
#include <sys/resource.h>

static double now(void) {
   struct timespec t; clock_gettime(CLOCK_MONOTONIC, &t);
   return t.tv_sec + t.tv_nsec / 1e9;
}
static double tv(struct timeval t) { return t.tv_sec + t.tv_usec / 1e6; }

/* copy src to dst with a heap buffer of b bytes; return number of read() calls */
long cp(const char *src, const char *dst, size_t b)
{
   char *buf = malloc(b);
   int in  = open(src, O_RDONLY);
   int out = open(dst, O_WRONLY | O_CREAT | O_TRUNC, 0644);
   long calls = 0; ssize_t n;
   while ((n = read(in, buf, b)) > 0) {
      calls++;
      if (write(out, buf, n) < 0) break;
   }
   close(in); close(out); free(buf);
   return calls;
}

int main(void)
{
   size_t sizes[] = {1, 16, 64, 512, 4096, 65536, 1048576};
   for (int i = 0; i < 7; i++) {
      struct rusage a, b;
      getrusage(RUSAGE_SELF, &a);
      double t0 = now();
      long c = cp("big.dat", "o.dat", sizes[i]);
      double real = now() - t0;
      getrusage(RUSAGE_SELF, &b);
      printf("%8zu %9ld  real %.4f  user %.4f  sys %.4f\n", sizes[i], c, real,
            tv(b.ru_utime) - tv(a.ru_utime), tv(b.ru_stime) - tv(a.ru_stime));
   }
   return 0;
}
```

The test files were created with `head -c 16777216 /dev/urandom > big.dat` (and a 1 MiB file for the one-byte and 16-byte runs).

A.2 Process versus thread creation

```
#define _GNU_SOURCE
#include <stdio.h>
#include <unistd.h>
#include <time.h>
#include <pthread.h>
#include <sys/wait.h>

static double now(void) {
   struct timespec t; clock_gettime(CLOCK_MONOTONIC, &t);
   return t.tv_sec + t.tv_nsec / 1e9;
}
void *worker(void *arg) { return arg; }

int main(void)
{
   double t = now();
   for (int i = 0; i < 1000; i++) {          /* 1,000 processes, one after another */
      pid_t p = fork();
```

```
        if (p == 0) _exit(0);
        waitpid(p, NULL, 0);
    }
    printf("1000 fork+wait: %.4f s\n", now() - t);

    t = now();
    for (int i = 0; i < 1000; i++) {            /* 1,000 threads, one after another */
        pthread_t th;
        pthread_create(&th, NULL, worker, NULL);
        pthread_join(th, NULL);
    }
    printf("1000 thread create+join: %.4f s\n", now() - t);
    return 0;
}
```

Compile with gcc -O1 -pthread -o procthread procthread.c.

Appendix B – Raw measurement output

B.1 Task 6 program (completed copy_file)

```
buffer    1 bytes: 10001 read() calls
buffer  512 bytes:    21 read() calls
buffer 4096 bytes:     4 read() calls
```

B.2 Task 7 program (job 1 and job 2)

```
shell: running job 1
42
job 1 exit status 0
shell: running job 2
job 2 exit status 3
```

Redirected output with the second fflush removed and exit(3) instead of _exit(3):

```
shell: running job 1
42
job 1 exit status 0
shell: running job 2
job 1 exit status 0
shell: running job 2
job 2 exit status 3
```

B.3 First benchmark run (single runs)

```
buffer(B)   file      reads      real    user    sys
       1   1 MiB    1048576   0.4928  0.1194  0.3718
      16  16 MiB    1048576   0.5276  0.1184  0.4049
      64  16 MiB     262144   0.1364  0.0282  0.1005
     512  16 MiB      32768   0.0423  0.0009  0.0231
    4096  16 MiB       4096   0.0193  0.0005  0.0097
   65536  16 MiB        256   0.0299  0.0003  0.0090
 1048576  16 MiB         16   0.0126  0.0004  0.0053
mmap copy 16 MiB:            0.0378 s
1000 fork+wait:              0.0531 s
1000 vfork+wait:             0.0086 s
1000 thread create+join:     0.0086 s
1000 fork+exec(/bin/true):   0.7024 s
getpid syscall: 106.3 ns each; trivial loop step: 0.34 ns
```

B.4 Repeated runs (7 runs per row, 16 MiB copy, median)

```
buffer(B)   median     min      max
      512   0.0376   0.0300   0.0583
     4096   0.0243   0.0184   0.0307
    16384   0.0111   0.0096   0.0240
    65536   0.0075   0.0067   0.0111
   262144   0.0071   0.0066   0.0092
  1048576   0.0075   0.0070   0.0082
```

```
1000 fork+wait (median of 7):              0.0527 s
1000 thread create+join (median of 7):     0.0077 s
1000 simultaneous threads (1 run):         0.0144 s
1000 simultaneous processes (1 run):       0.1098 s

fork+exit+wait with parent memory touched (200 repetitions):
   1 MiB   83.4 us
   8 MiB  156.1 us
  64 MiB  763.6 us
 512 MiB 4959.9 us
```

B.5 Test platform

Linux 6.18 on x86-64, one virtual CPU, sandboxed environment, gcc, no strace.

Appendix C – How to reproduce the measurements and add the strace evidence

Run these commands on any Linux machine and paste the results into the places marked in the report.

Purpose	Command	What to look for
Compile Task 6	gcc -O2 -o copy copy.c && ./copy	The three lines of output (10001, 21, 4)
Count system calls	strace -c ./copy	In the table, the calls column for read should be about 10,027 and for write about 10,027–10,029 (derived in Section 7.5)
See individual calls	strace -e trace=openat,read,write -o trace.txt ./copy	Open trace.txt: read(3, ..., 1) = 1 repeated, then read(3, ..., 512) = 512, then read(3, ..., 4096) = 4096
User and system time	time ./copy	real, user, sys; the sys share is high for tiny buffers
Trace the shell program	strace -f -e trace=process ./shell	clone/fork, execve("/usr/bin/expr", ...), wait4, exit_group
Redirect experiment	./shell2 > out.txt; cat out.txt	Duplicate "shell: running job 2"
Process versus thread	gcc -O1 -pthread procthread.c -o procthread && ./procthread	The 6–8 times ratio
Repeat for stability	run each benchmark 5–7 times and take the median	Reduces noise

Evidence to paste (to be completed on your own machine):

- strace summary for the Task 6 program: *Insert your strace output.*
- strace trace of three or four consecutive read() calls: *Insert your strace output.*
- strace of the shell program showing fork/clone, execve, wait4: *Insert your strace output.*

Appendix D – Evidence checklist against the task sheet

Task	Evidence expected	Where delivered in this report	Status
1	Ranked requirement table for every environment, with justification	Section 2.3	Done

Task	Evidence expected	Where delivered in this report	Status
1	Alternative analysis: one OS on every device (alarm delay, memory, administration)	Section 2.7	Done (estimates labelled)
2	Justified OS type for each environment with rejected alternatives	Section 3.2	Done
2	SMP versus AMP; cluster fail-over; single multiprocessor alternative	Sections 3.3–3.5	Done
3	Structure comparison and defended choice for two environments	Sections 4.2–4.4	Done
3	Monolithic versus microkernel on monitors	Section 4.5	Done
4	Service table for each environment	Section 5.2	Done
4	Services removable from monitor; essential for database; record protection	Sections 5.3–5.5	Done
4	Full desktop OS on tablets	Section 5.6	Done
5	Step-by-step system call with strace evidence	Sections 6.3–6.5	Explained; **strace output to be pasted** (strace not installed on test machine)
5	fopen/fread/fwrite versus system calls	Section 6.6	Done
6	Completed copy_file and its output	Section 7.4	Done, **real output**
6	Measured system-call counts and times; recommendation	Sections 7.6, 11.2–11.3	Done
6	mmap alternative	Section 7.7	Done, measured
7	Working shell-like program, diagrams, reasoned evaluation	Sections 8.2, 8.5–8.7	Done, **real output**
7	Flush experiment explained	Section 8.6	Done, **real output**
8	CPU modes, interrupts, protection applied to monitors	Sections 9.1–9.4	Done
8	Polling versus interrupts	Section 9.5	Done (estimates labelled)
9	Data-placement plan and I/O method for each device	Sections 10.2–10.3	Done
9	DMA versus interrupt per word	Section 10.5	Done
10	Measured results in tables, compared with expectations	Sections 11.2–11.6	Done
10	Threads versus processes	Sections 11.4, 11.5, 11.7	Done, measured
11	Complete plan, risks, three-hospital scalability, virtual machines	Sections 12.1–12.5	Done
12	A. Impact analysis; B. Evaluation; C. Adaptation; D. Defence	Sections 13.1–13.5	Done

Appendix E – Glossary

Term	Meaning
AMP / SMP	Asymmetric / symmetric multiprocessing
Amdahl's law	Speed-up limit of parallel work: $S(N) = 1 / ((1 - p) + p/N)$

Term	Meaning
Availability	Fraction of time a system works; MTBF / (MTBF + MTTR)
Buffer cache / page cache	Part of main memory holding recently used disk data
Cluster	Computers that cooperate to give one service and take over from each other
Copy-on-write	Pages are shared until one process writes, then copied
DMA	Direct memory access: a controller moves data to memory without the CPU copying each word
Exit status	Small integer a process returns when it ends; 0 means success
Fail-over	Automatic switch of a service to a standby node
Hard real-time	A missed deadline is a failure
Hypervisor	Software that runs virtual machines
Interrupt / trap	Hardware / software event that transfers control to the kernel
Kernel mode / user mode	Privileged / restricted CPU mode (mode bit)
Microkernel	Kernel with only minimal functions; services run as user processes
MMU / MPU	Memory-management unit / memory-protection unit
Orphan / zombie	Running process whose parent has ended / finished process not yet waited for
OTA update	Over-the-air (remote) software update
RTOS	Real-time operating system
Split-brain	Both nodes of a cluster believe they are the active one
System call	Request from a program to the kernel for a service
TLB	Cache of recent address translations
Zero trust	Security model that authenticates every request and does not trust the network location

Appendix F – Summary of what was measured and what was assumed

Kind of statement	Examples
Measured in this study	System-call counts (10,001 / 21 / 4 and the n/b table); copy times with different buffer sizes; user and system CPU time; 106 ns null system call; mmap copy time; fork, vfork, thread and fork+exec creation times; effect of parent memory size on fork
Derived by calculation from stated assumptions	Availability figures, Amdahl speed-ups, battery life, interrupt counts for DMA versus per-word interrupts, effective access time with a cache, sizing of the home-monitoring load
Typical values from the literature (not measured here)	Memory-hierarchy access times; microkernel message cost; interrupt latency of an RTOS; internet round-trip times; virtualisation overhead; per-process and per-thread memory
Not available on the test machine	strace output; real-time hardware; a microkernel; multi-processor scaling (the test machine had one virtual CPU)